Database Design

Summary

a) **Conceptual Database Design**
   - The E-R Model

b) **Design principles and Methodologies**
   - Life cycle of databases within Information Systems
   - Quality of conceptual models

c) **Logical database Design**
   - Translating from the E-R model into the Relational Model

d) **Normalization**
   - Functional dependencies and normal forms

e) **Physical Database Design**
   - (hints to) Access structures, Indexing, Query plans

Conceptual Database Design

a) **Conceptual Database Design**
   - The E-R Model

b) **Design principles and Methodologies**
   - Life cycle of databases within Information Systems
   - Quality of conceptual models

c) **Logical database Design**
   - Translating from the E-R model into the Relational Model

d) **Normalization**
   - Functional dependencies and normal forms

e) **Physical Database Design**
   - (hints to) Access structures, Indexing, Query plans

ABSTRACTIONS

in Conceptual Models for Databases

The basic ingredients of Conceptual Models

- **Abstraction**: the act (or capability) of identifying and pointing out properties that are shared by sets of objects

- Three **distinct** abstractions are at the foundation of knowledge representation:
  - Classification
  - Aggregation
  - Generalization

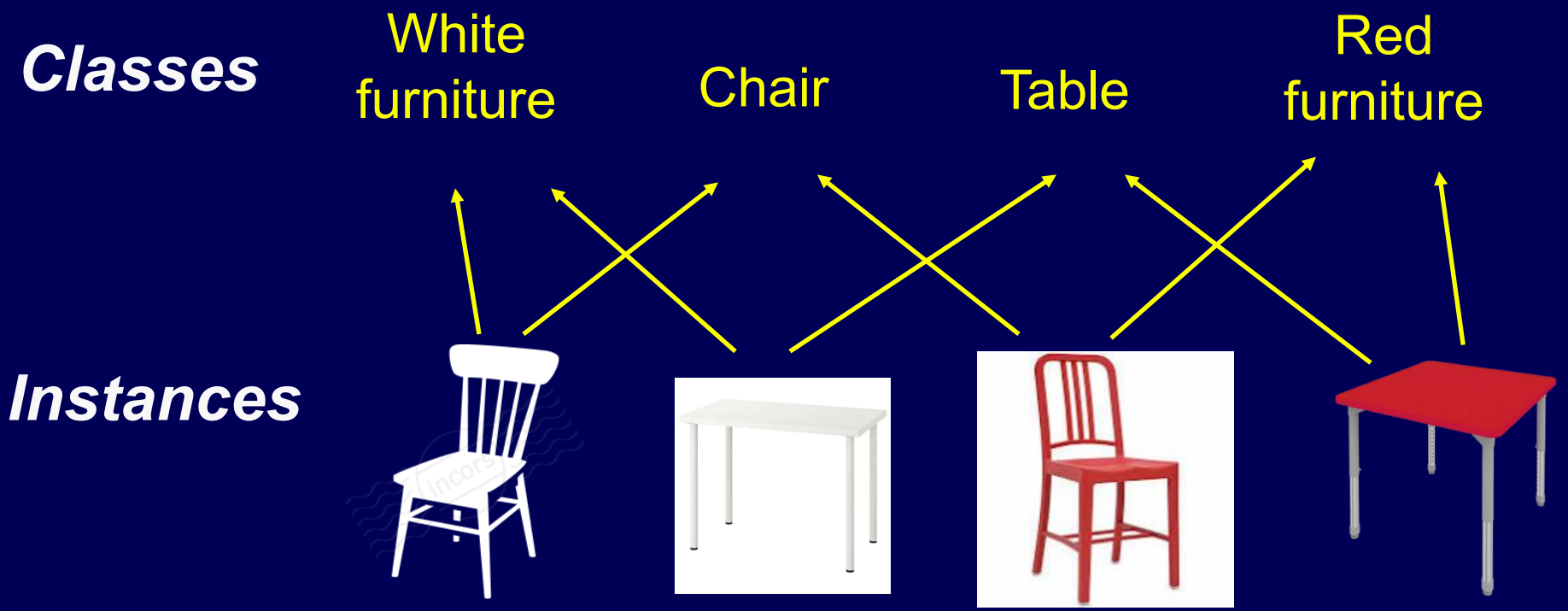
Classification

The act (or capability) of defining classes of items taken from the real world (either objects or facts)

- Books
- Motorbikes
- Persons
- Contracts
- …

Each class has an (implicit) «membership test» to tell whether an item of the real world is an instance of the class or not

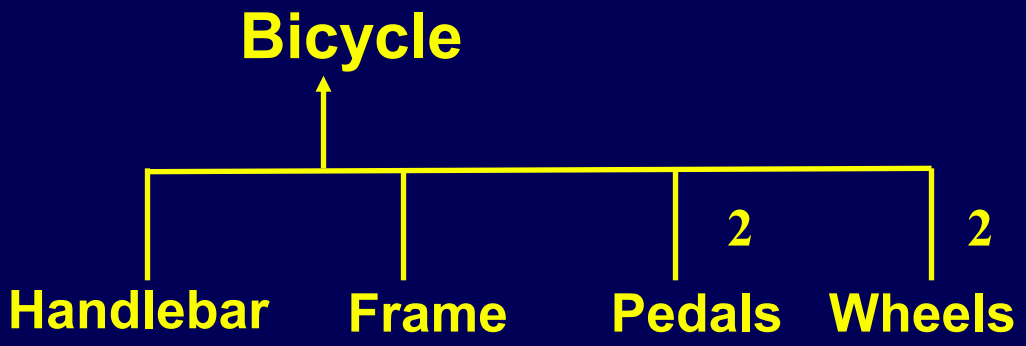
Classification (with multiple membership)



Aggregation

The act (or capability) of building a complex class by aggregation of simpler classes (called *components*)

- Bicycle:
  - Wheels
  - Pedals
  - Frame
  - Handlebar

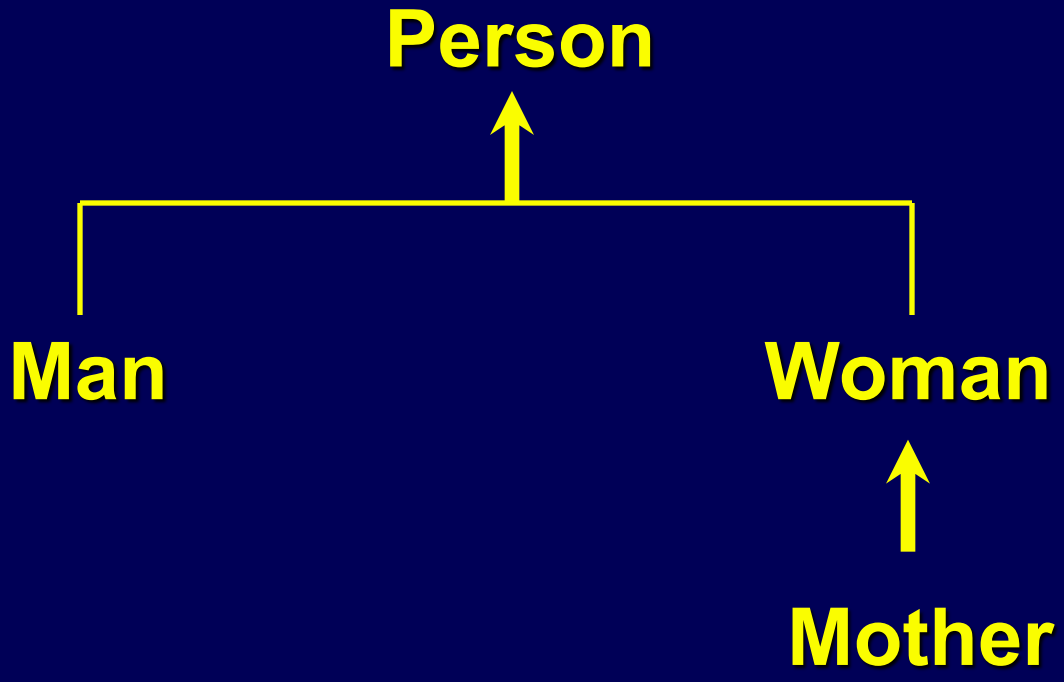


Each component is annotated with the number of instances required to build an instance of the aggregated class (default: 1)

Generalization

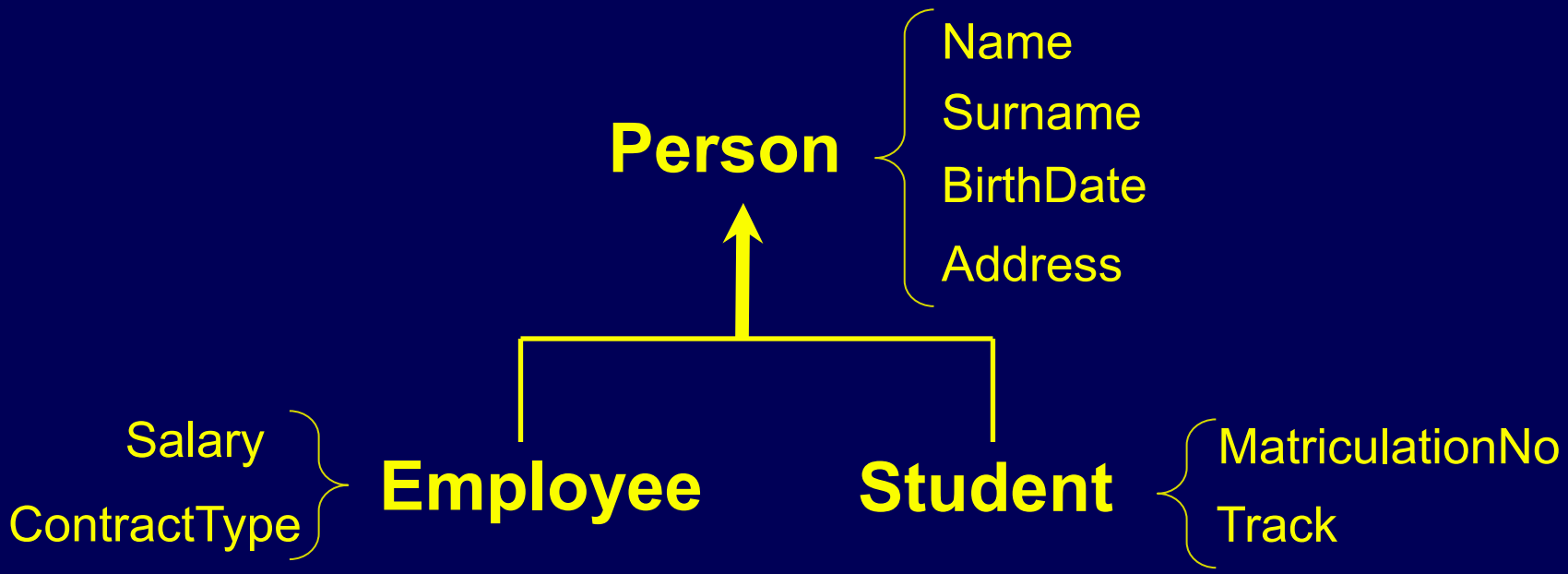
Establishes «subset» relationships

- Mother ⊂ Woman ⊂ Person
- Man ⊂ Person



Inheritance with Generalization

Subsets inherit all the properties of their supersets



The E–R model

(ENTITY–RELATIONSHIP)

E-R Model

- E-R: Entity-Relationship
  - Proposed by Peter Chen in 1976
  - Well established as an industrial standard in most conceptual database design tools and methodologies
- Defines
  - *Entities* (classes of items) and
  - *Relationships* (associations between the entities)

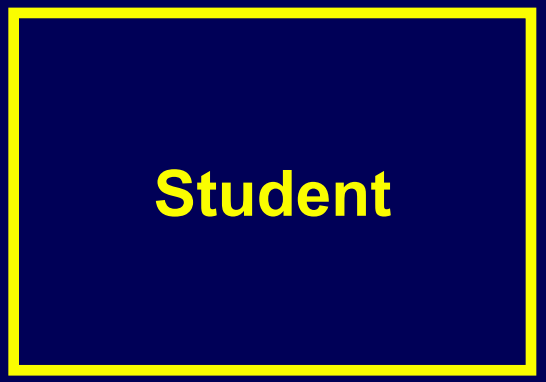
Entity

- An Entity represents a class of items
  - either **objects (**such as cars, students, employees, …)
  - or **facts** (such as bank accounts, university courses, contracts, …)
- The items must be relevant to the application
- In a model, each entity has a *unique name*

Visual representation of Entities



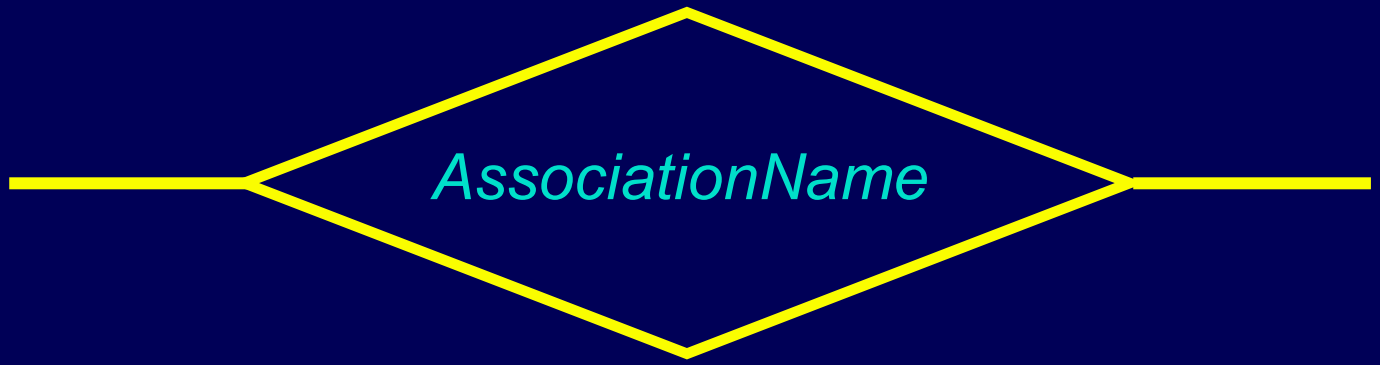
example:



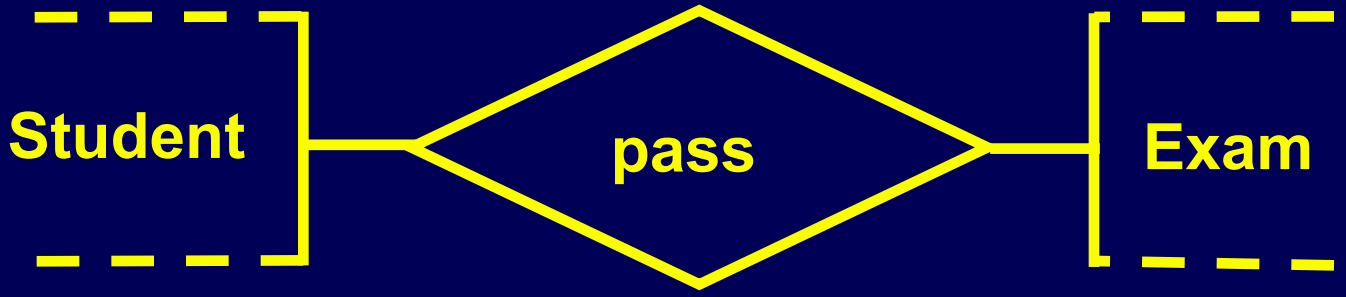
Relationships (or Associations)

- Express and represent a **logical binding** between entities that are relevant to the application
- Each instance of association is a n-uple of instances of entities
  - An association between n entities $E_1 \ldots E_n$ can be expressed as $R = \{<e_1, \ldots, e_n>\} \subseteq E_1 \times \ldots \times E_n$
  - e.g., the binding between a car *c* and its owner *o* is represented by means of the couple $<c,o>$
- Each association has a unique name in a model

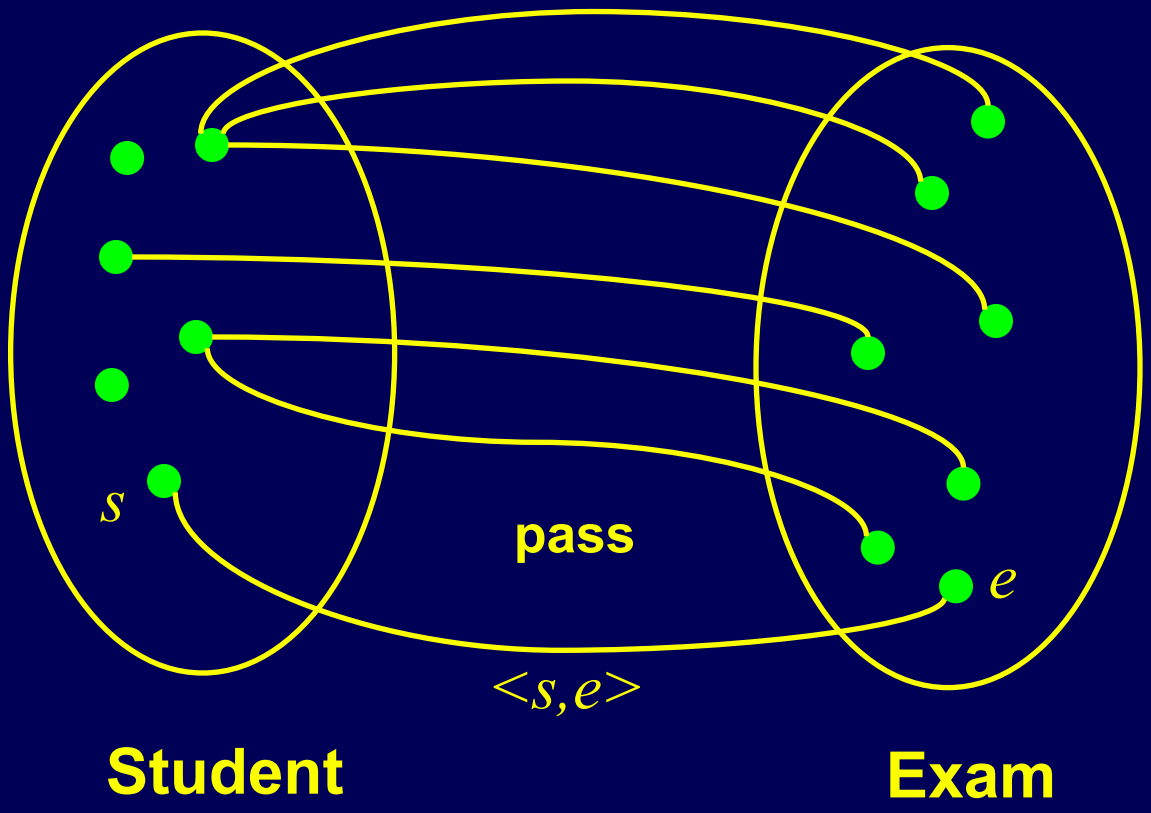
Visual representation of Relationships



Example:



Visual representation of the instances



Attributes and their visual representation

- Express and represent characteristics of Entities and Relationships that are relevant for the application
- Each attribute has a name (unique within the entity)



Guidelines for a correct design

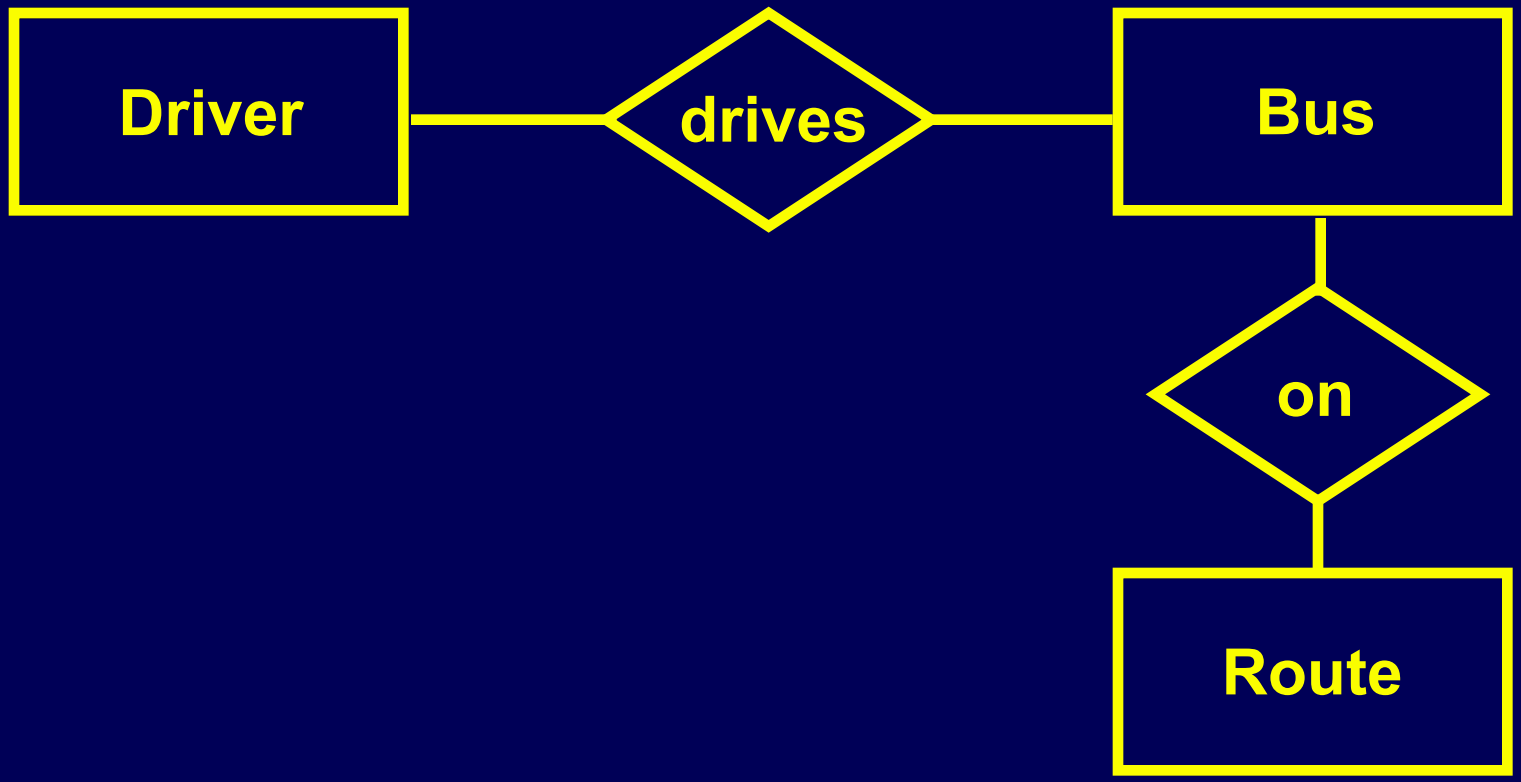
- If a concept is intrinsically relevant to the application context: **Entity**
- If a concept can be expressed with one elementary measure/value: **Attribute**
- If a concept defines/expresses a binding between two entities: **Relationship**

Mapping between concepts and E-R constructs

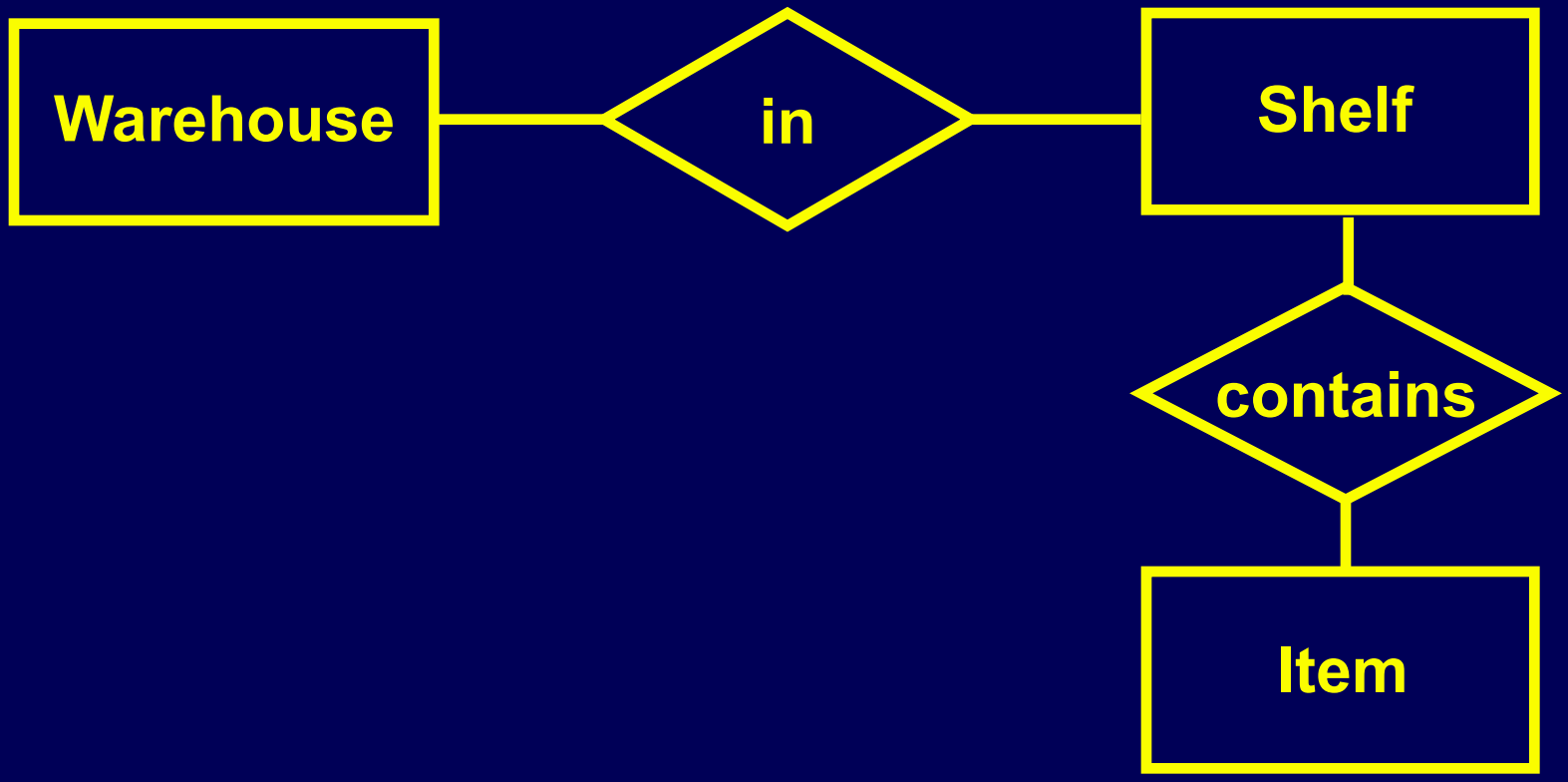
The mapping between
Objects and Facts of the real world
and
Entities, Relationships, and Attributes
is strictly context-dependent, not pre-defined:

- Example:
  - The **Color** of the car with plate n. EA450PF is *red*
    - Color is an attribute of entity Car, with value «red»
  - **Color** red has a wavelength equal to ~700 nm
    - *red* is an instance of entity Color, with its own attributes

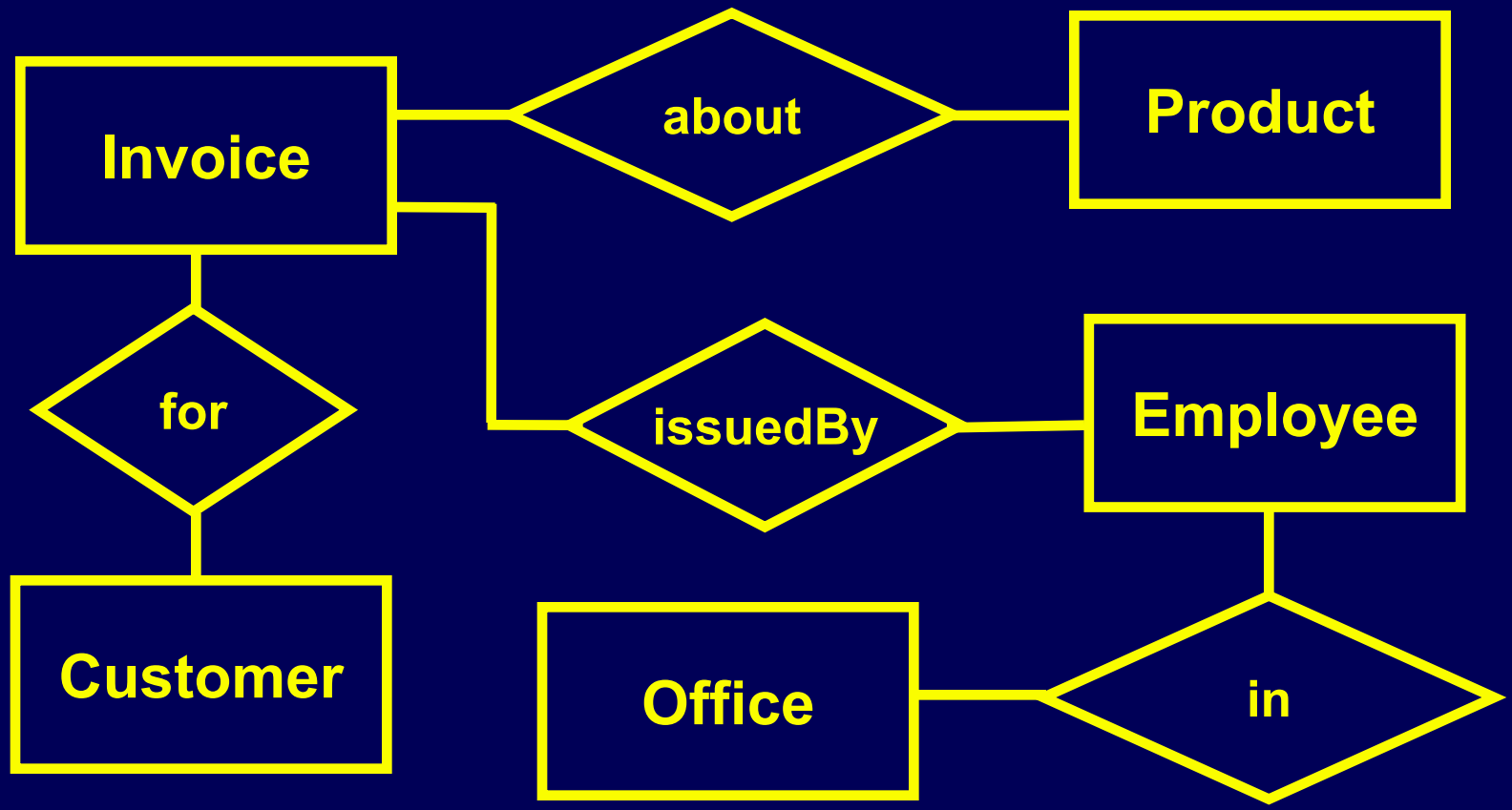
Public transports



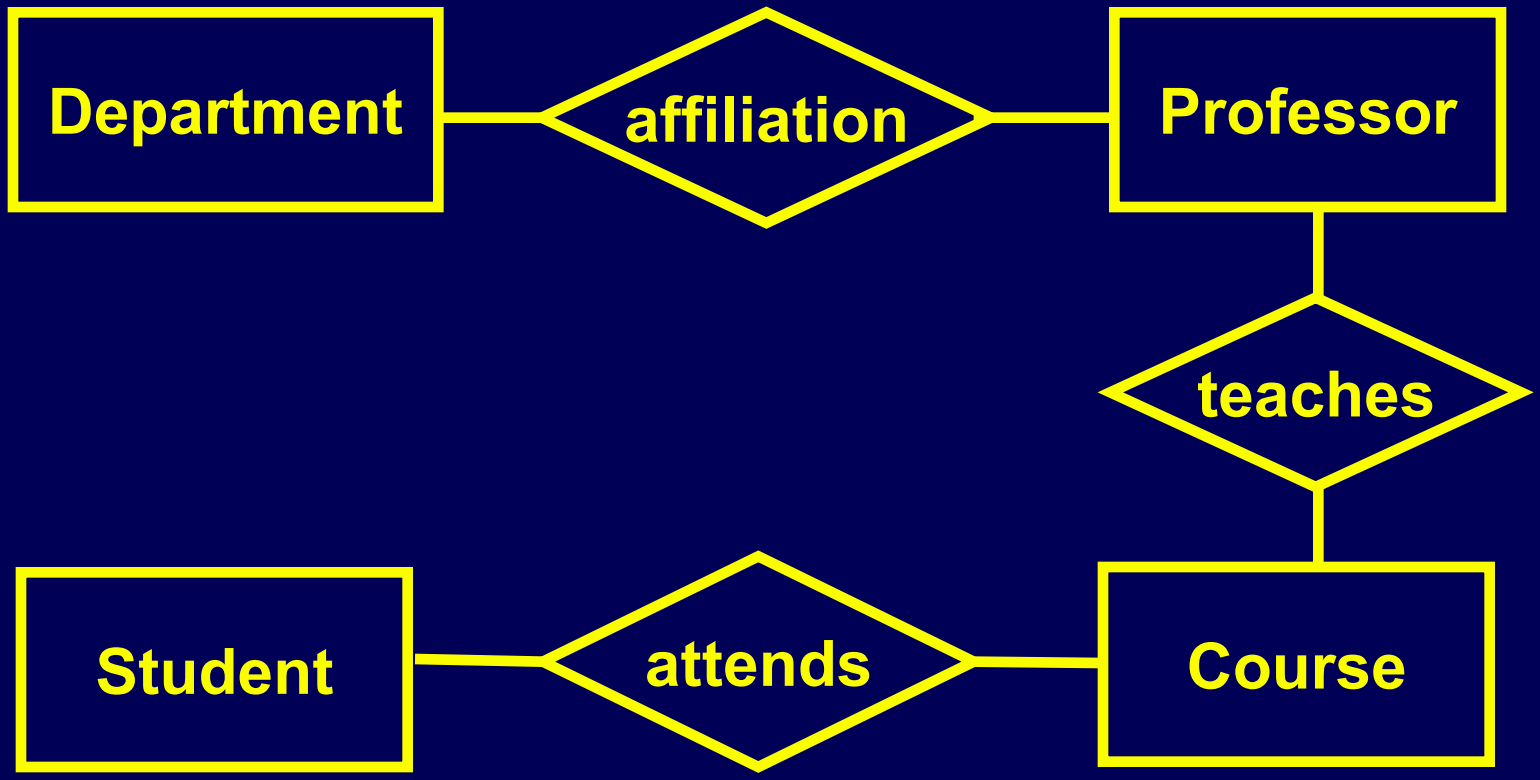
Warehouse management



Invoices


Invoice
about
Product
for
issuedBy
Employee
Customer
Office
in

University


Department
affiliation
Professor
teaches
Student
attends
Course

Relationships

Cardinality of associations

- In the general case, an instance *e* of entity *E* can take part to the same association A an arbitrary number of times
- The model requires that the constraints on the min and max number of «participations» to each association by an instance of an entity be explicit
- The constraints are expressed as couples: (MIN-CARD, MAX-CARD)

  MIN-CARD = 0 (optional)

  = 1 (mandatory)

  = *specific values* other than 0 or 1 are possible, but rare

  MAX-CARD = 1 (one)

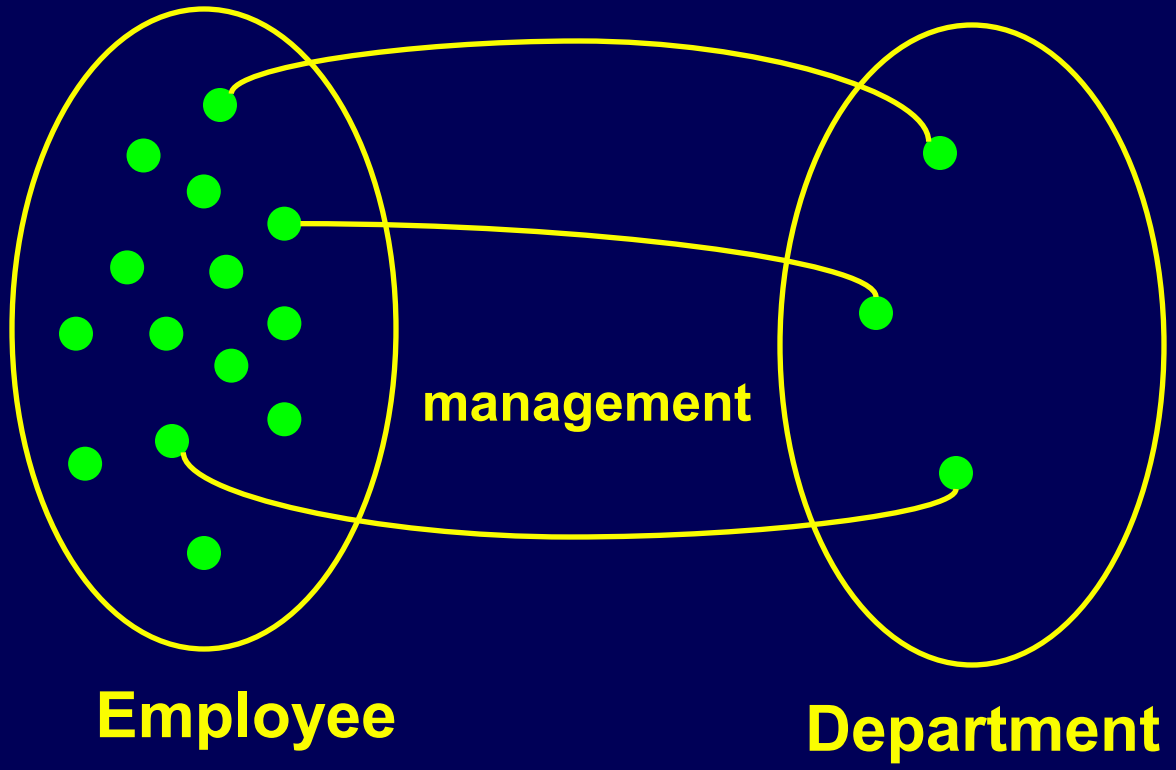
  = N (many)

  = *specific values* other than 1 or N are possible, but rare
- Based on the max cardinality expressed on each of their «sides», associations are classified as *one-to-one*, *one-to-many*, and *many-to-many*

one-to-one associations



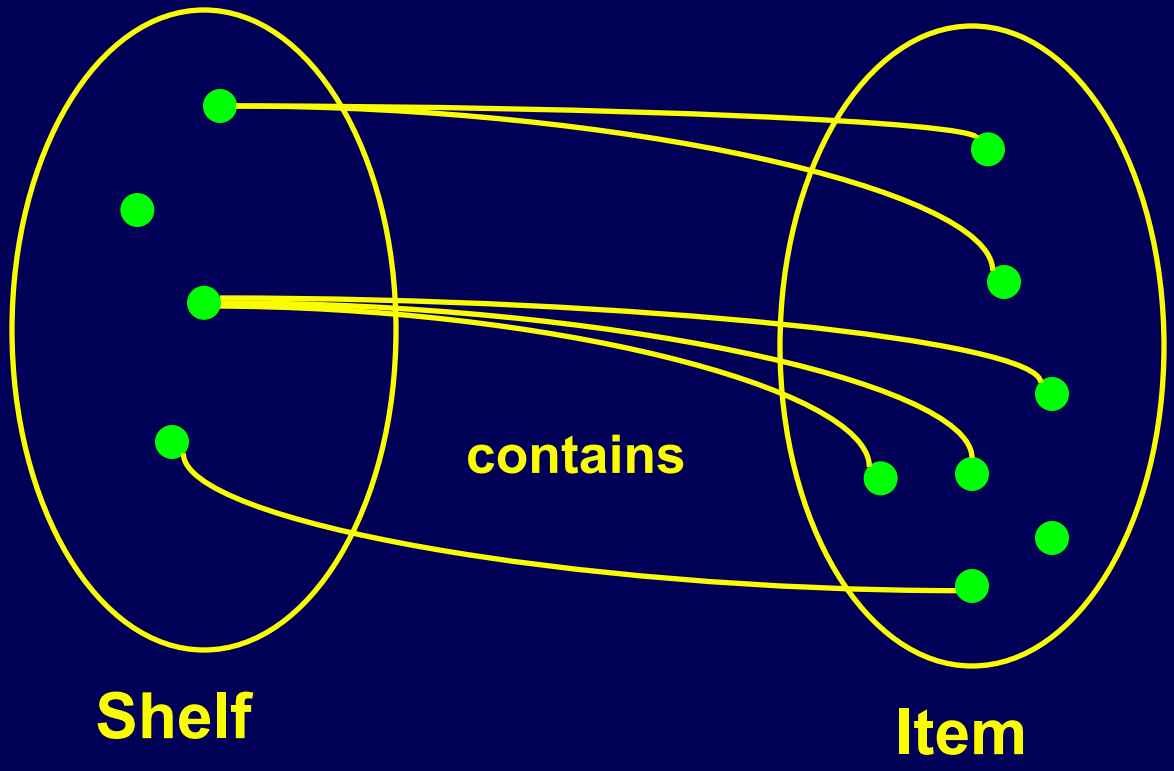
- **A department must have exactly one manager (1,1)**
- **An employee may be the manager of at most one department (0,1)**


management
Employee
Department

one-to-many associations



An item can be at most in one shelf (0,1)
A shelf can be empty or filled with many items (0,n)

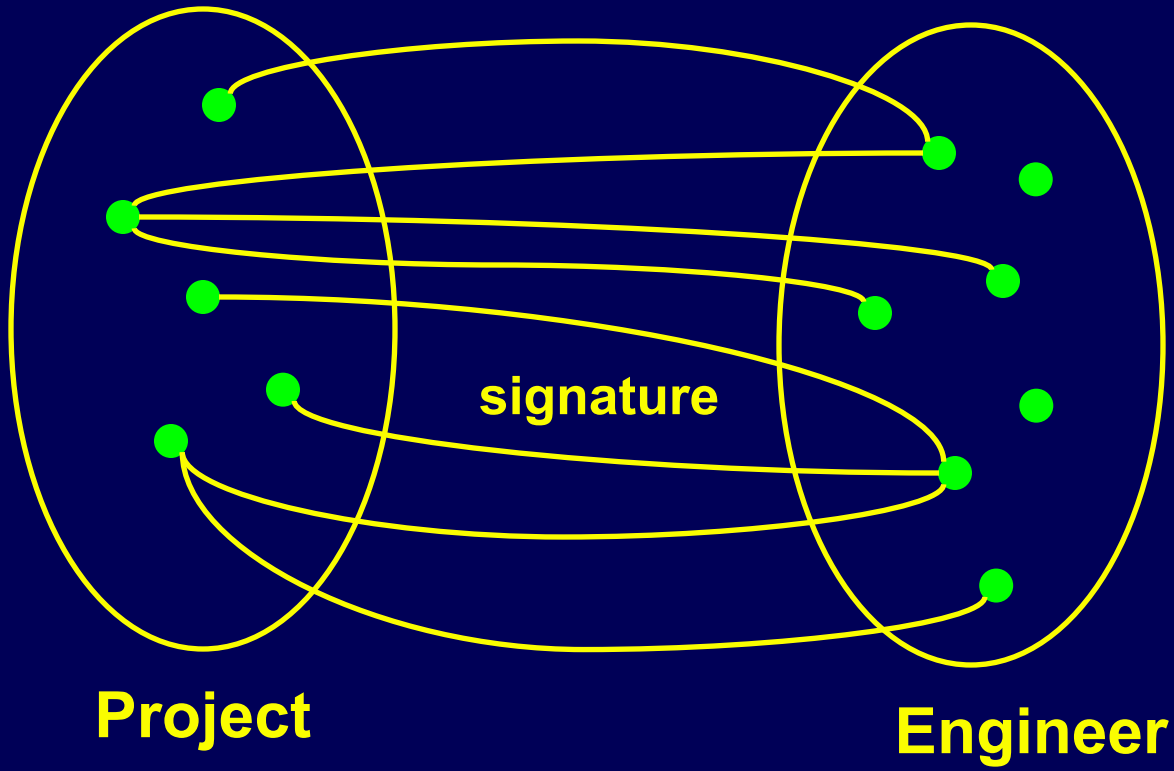

contains
Shelf
Item

many-to-many associations



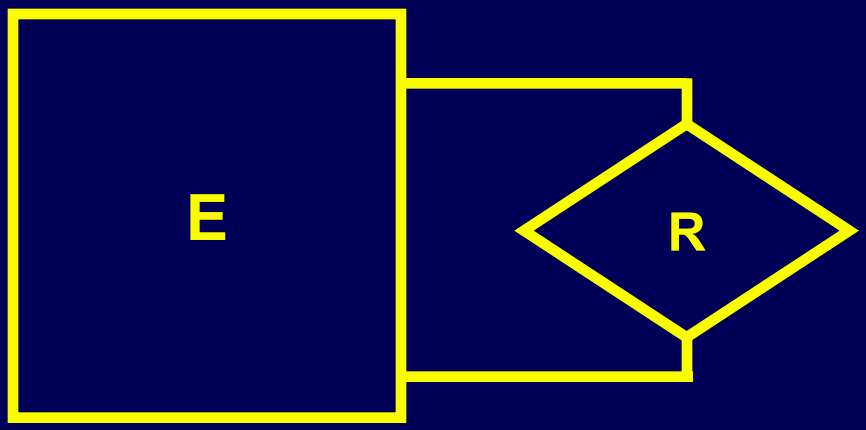
- **A project must be signed by at least one engineer (1,n)**
- **An engineer may have signed any number of projects (0,n)**

Here **n** is used twice, but in just means that that in both cases the maxcardinality in «unbound», not that they have the same max


signature
Project
Engineer

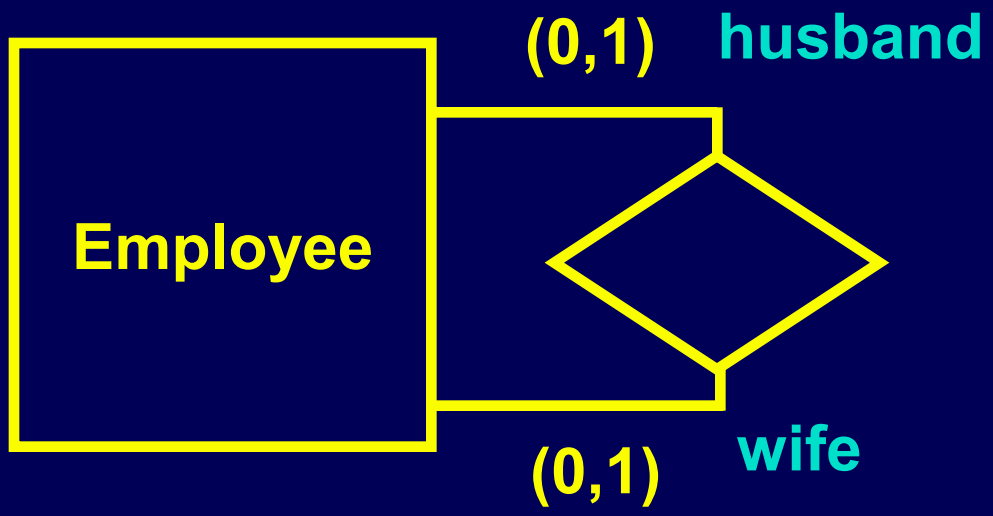
Self-associations

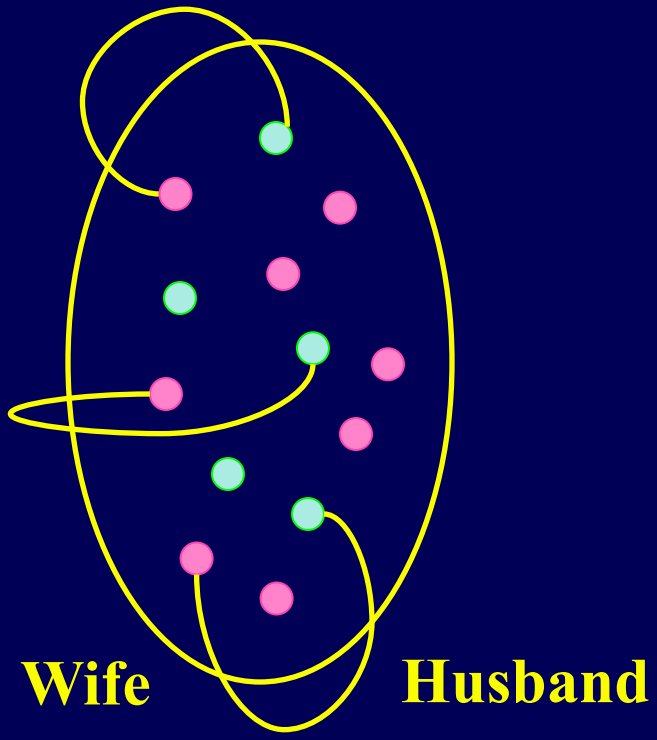
- Associations can also occur between items that are instances of the **same entity**
  - also called “recursive associations”

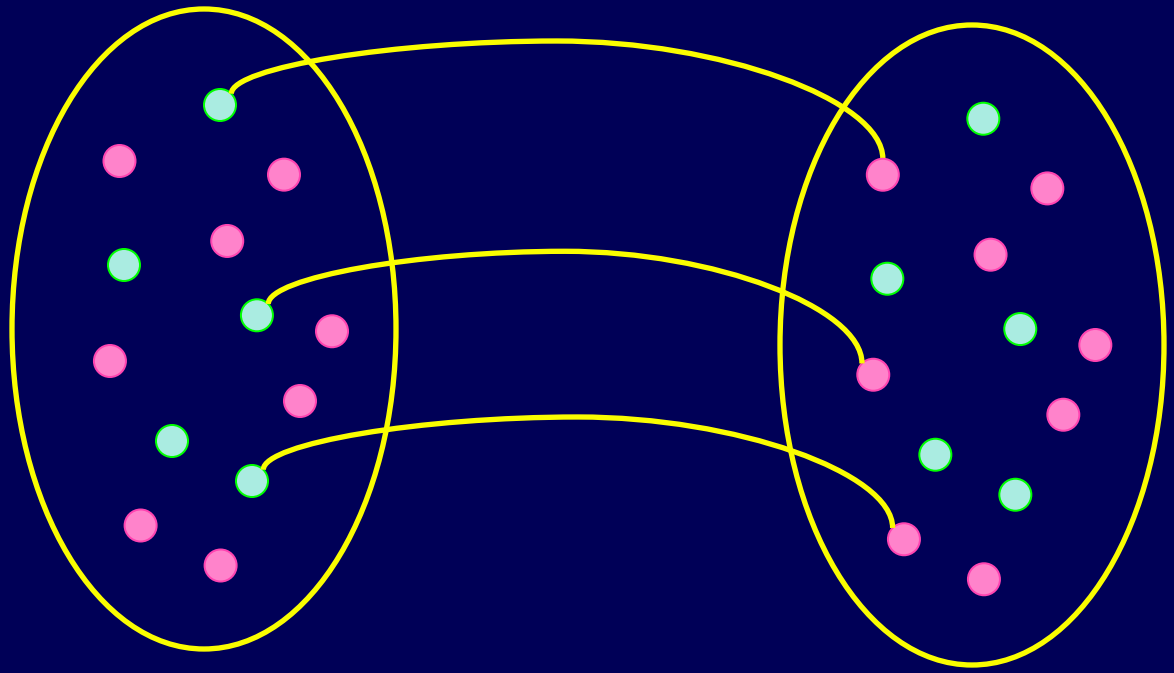


one-to-one self-associations

The sides of the association can be annotated with the "role" of the instances in the relationship



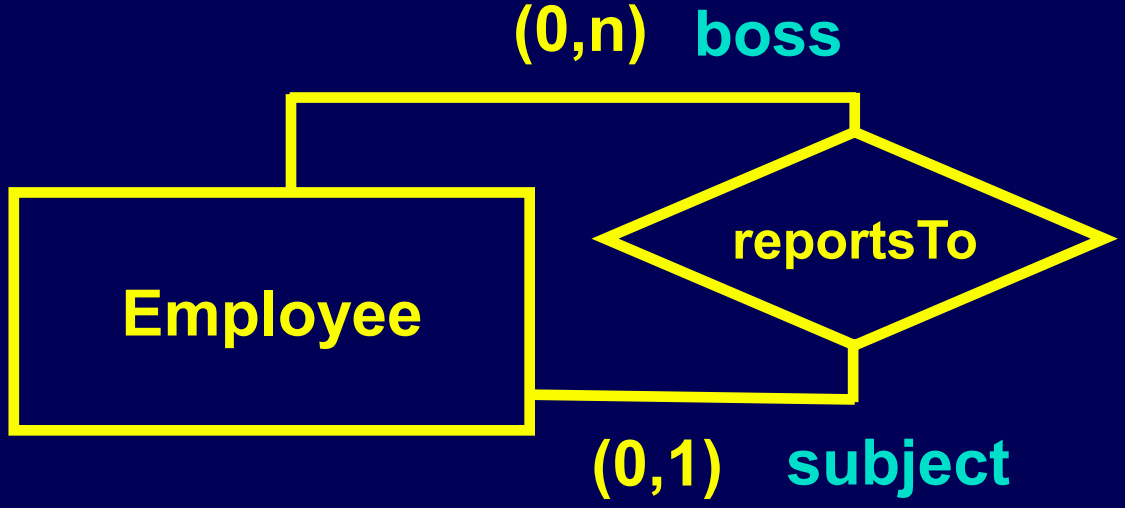

Wife
Husband



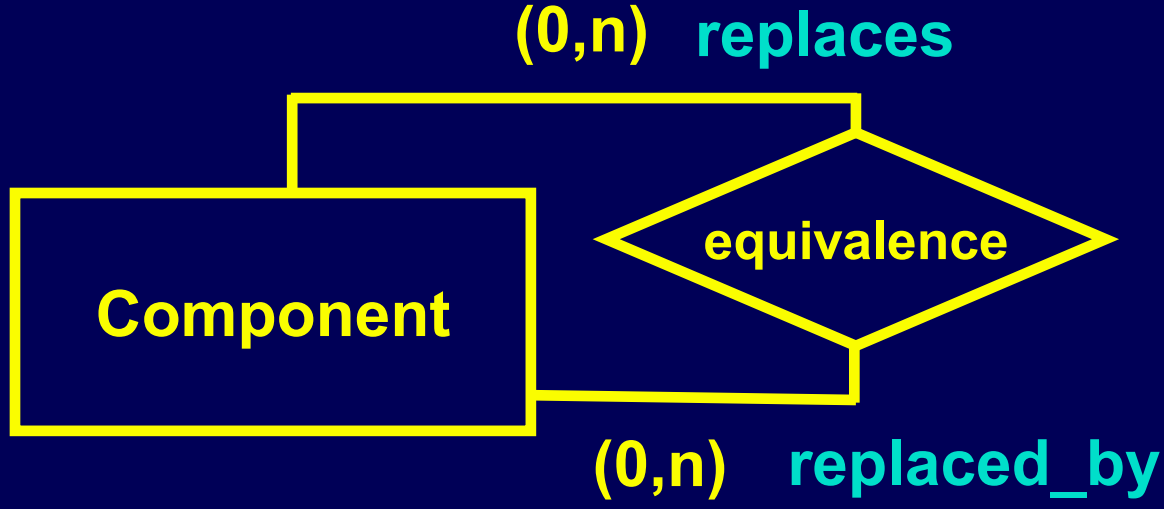
Husband

Wife

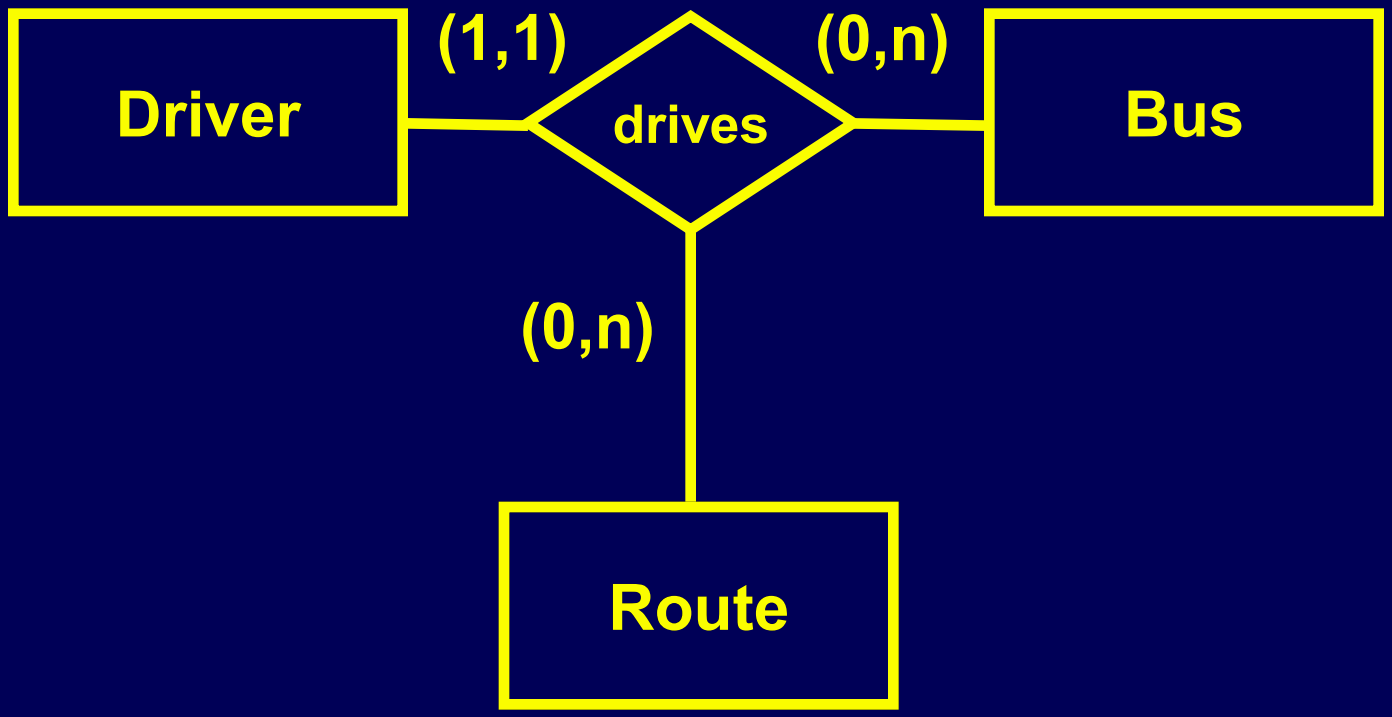
one-to-many self-associations

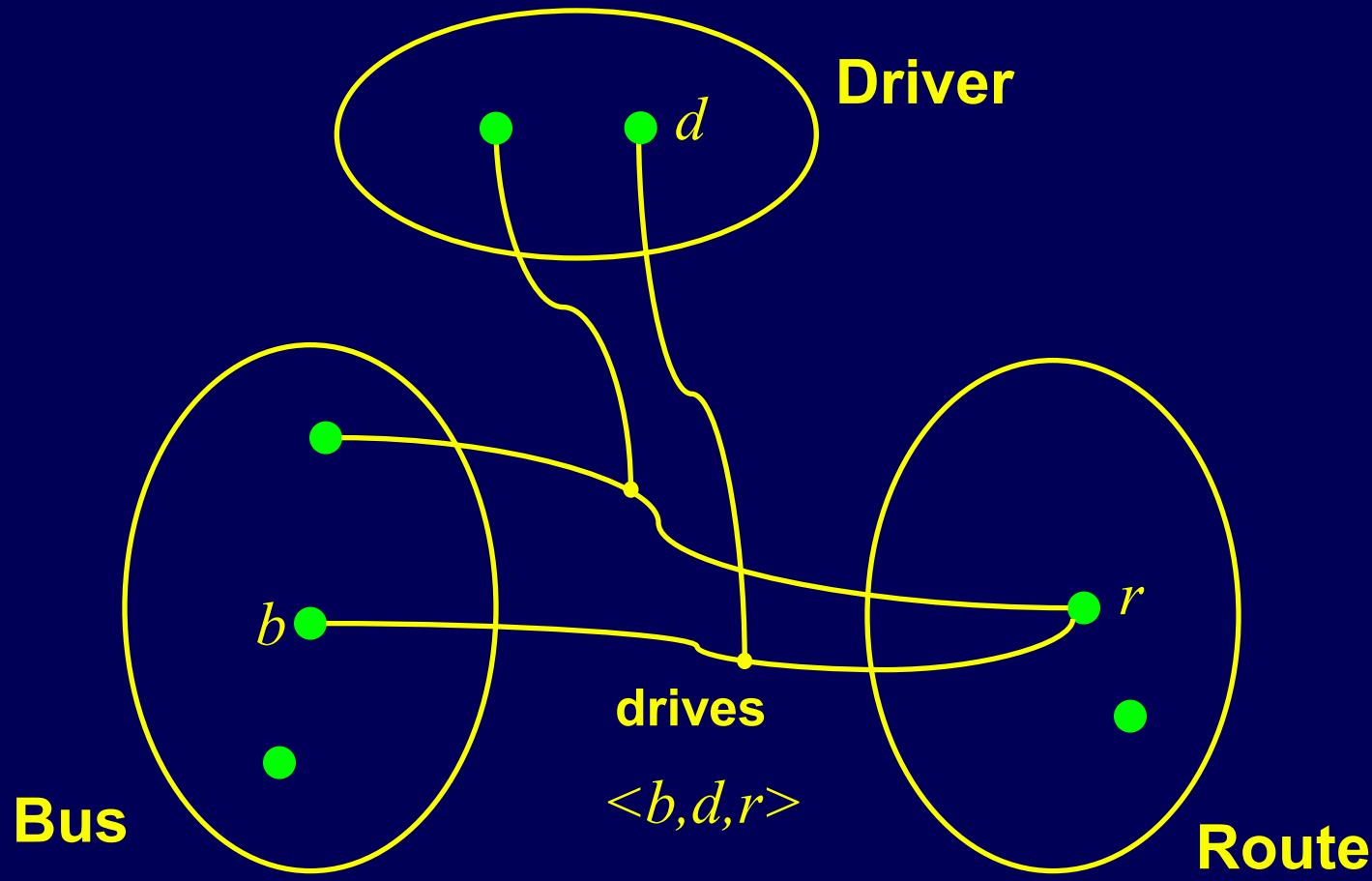


many-to many self-associations

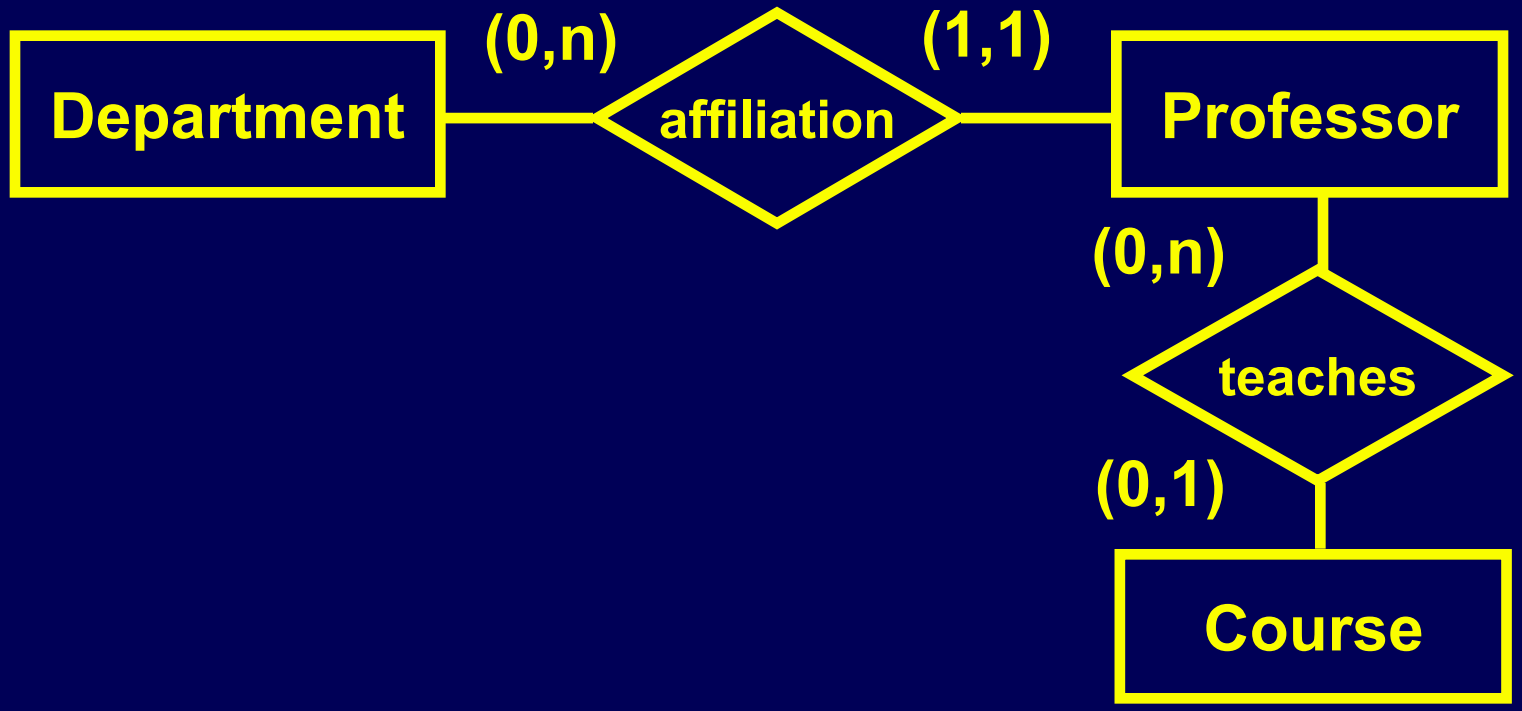


Ternary associations

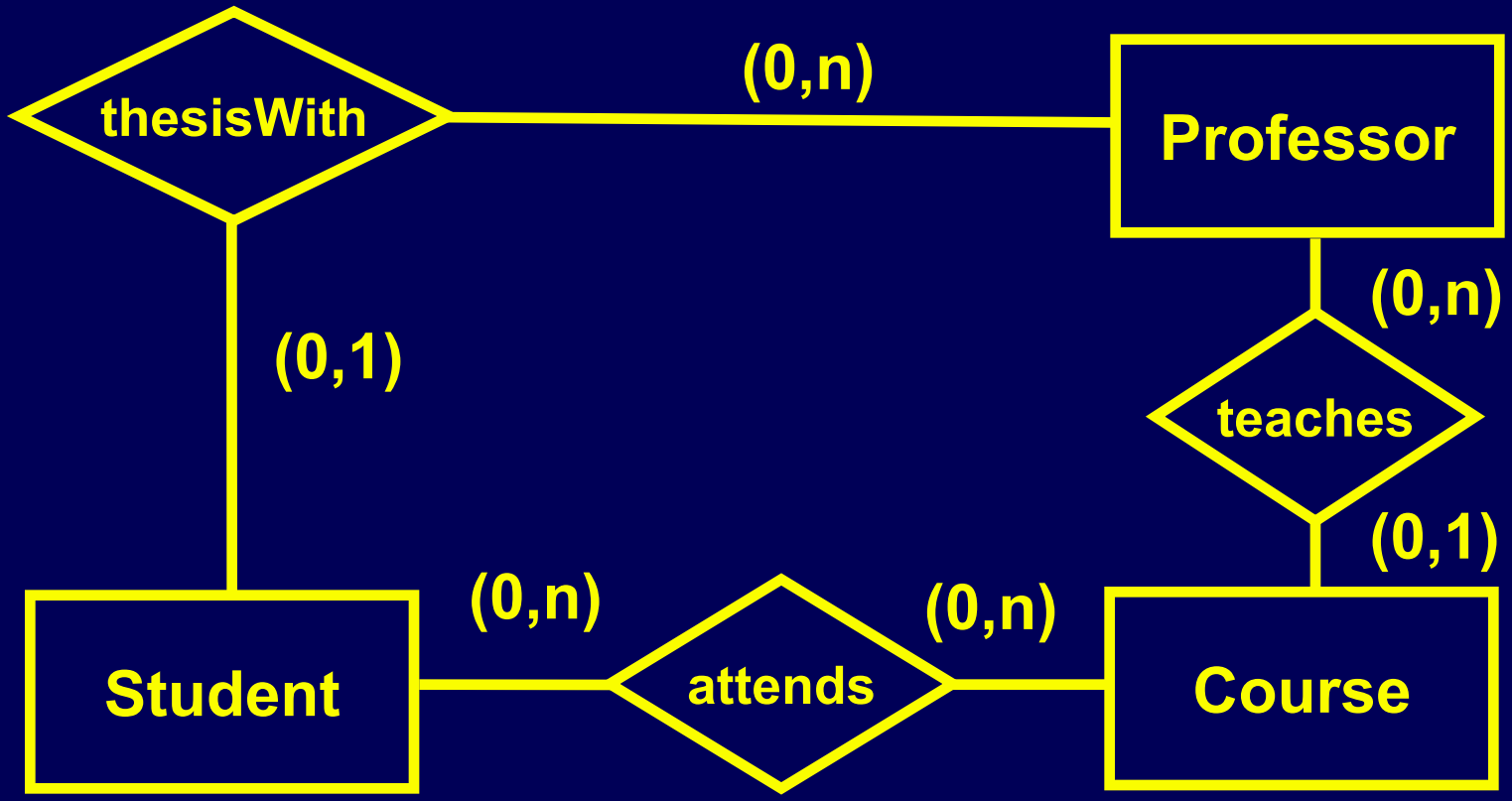



Driver
d
b
r
drives
<b,d,r>
Bus
Route

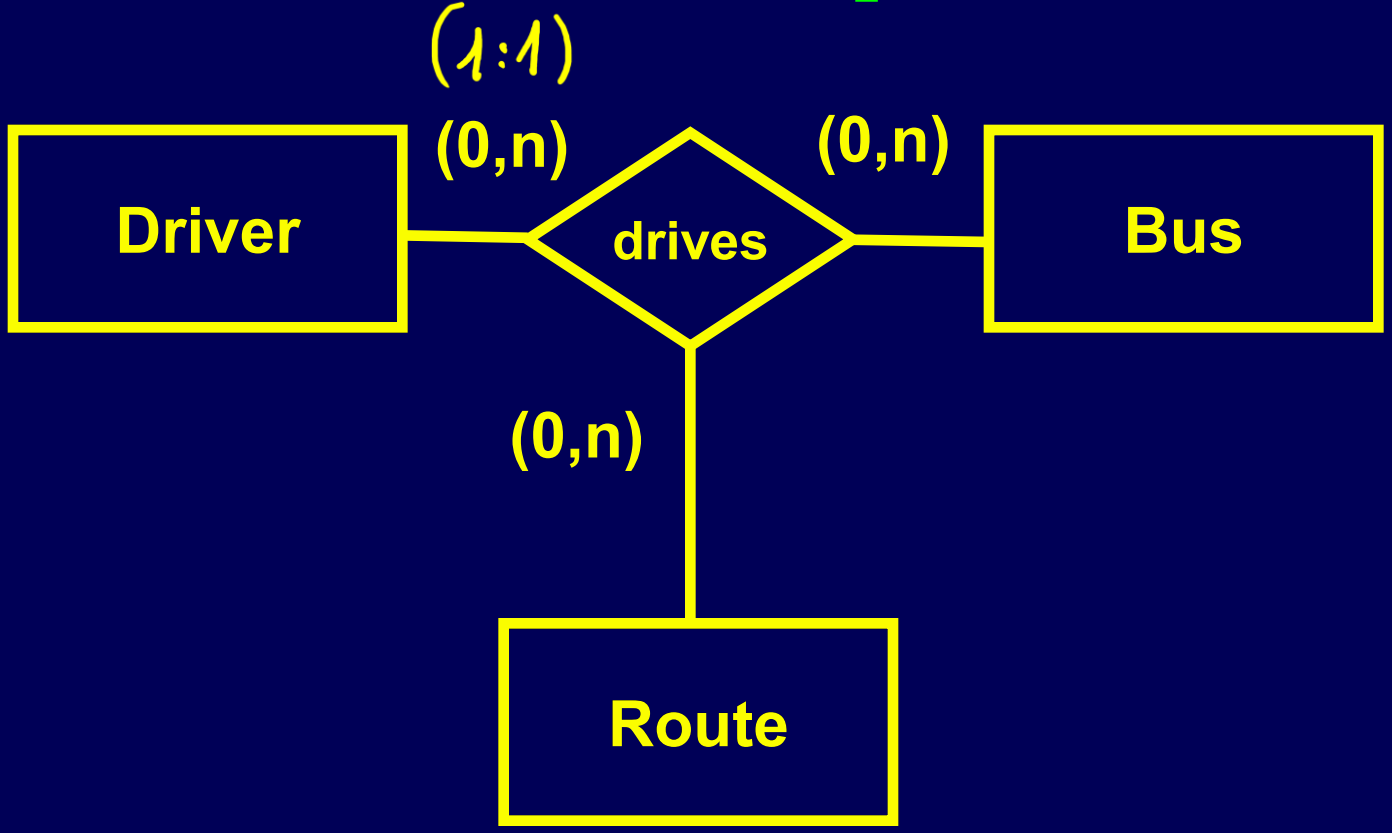
University


Department
(0,n)
affiliation
(1,1)
Professor
(0,n)
teaches
(0,1)
Course

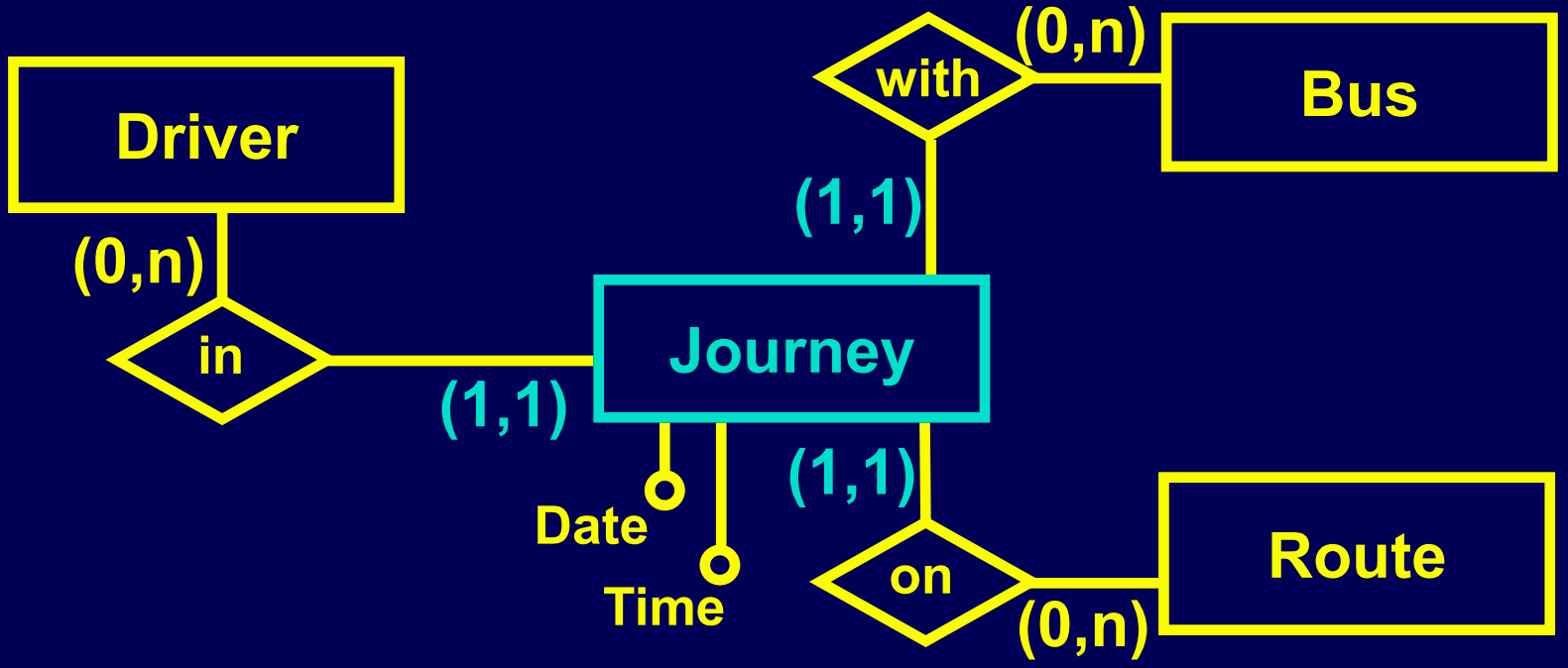
University


thesisWith
(0,n)
Professor
(0,1)
(0,n)
teaches
(0,1)
Student
(0,n)
attends
(0,n)
Course

Public transport

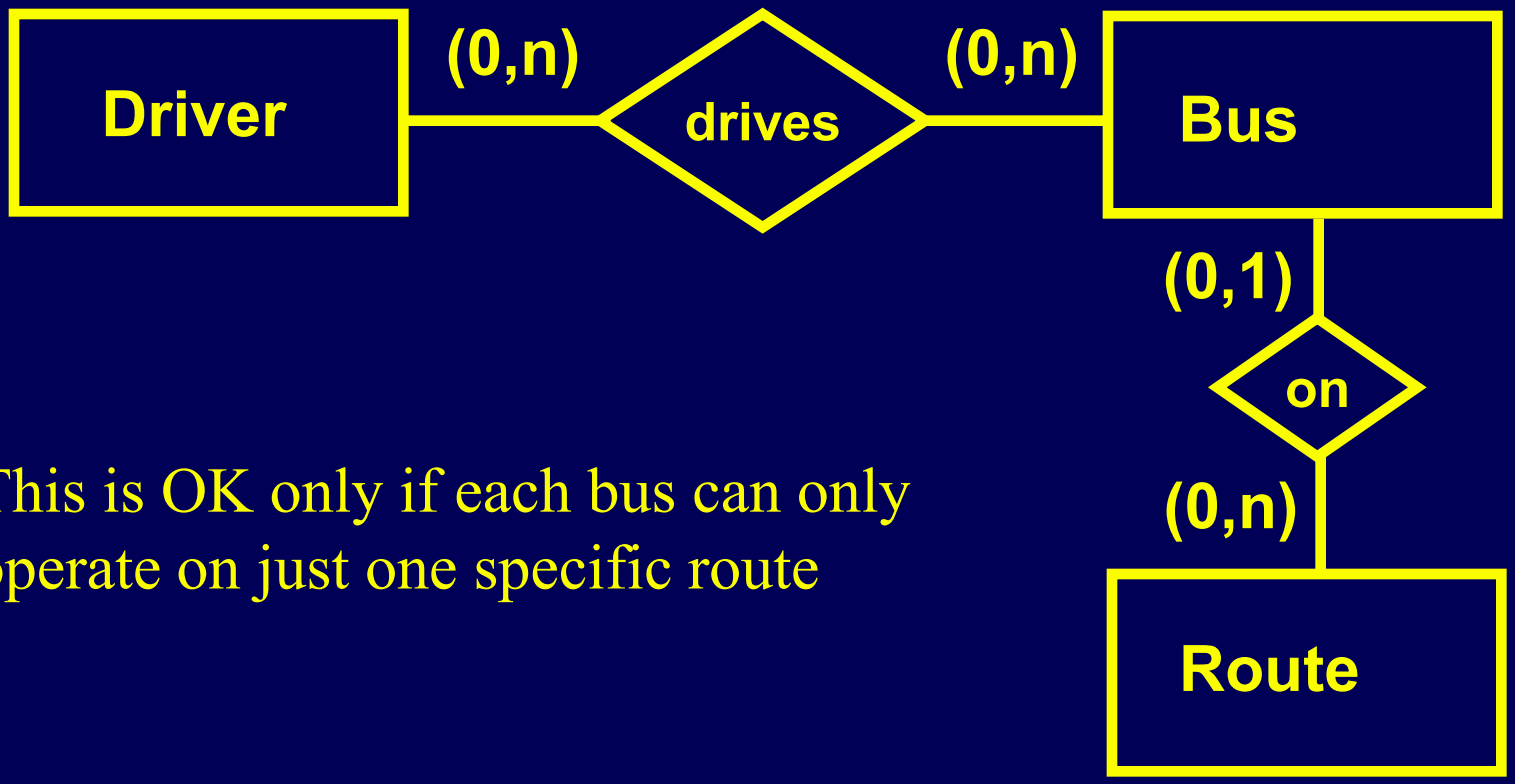


Public transport



- **Journey** is necessary (as an entity) if the same driver *d* can drive the same bus *b* on the same route *r* more than once (of course in distinct dates/times).
  - This is because it would be impossible to model the multiple, duplicated «participations» of the triplet <*d*,*b*,*r*> to the same relationship (that is modeled as a subset of a Cartesian product)

Public transport



This is OK only if each bus can only operate on just one specific route

Attributes and Identifiers

Cardinality of Attributes

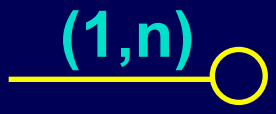
- Preliminary classification (4 types):

  Scalar attribute (simple, with one value)



e.g.: StudentId, Surname, Grade

Multiple attribute (can have many values)

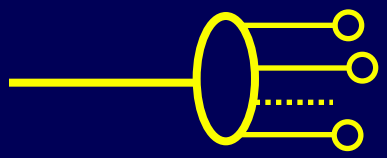


e.g.: phoneNo, diploma, skill

With (min,max) we also express the cardinality of attributes
If not specified, the default is assumed to be (1,1)

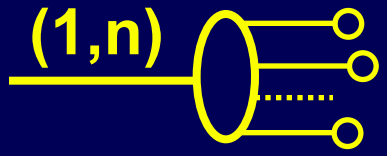
Composite attributes

Composite attribute



e.g.: Date (dd, mm, yyy),
Address (city, street, number, zipcode)

Composite Multiple attribute

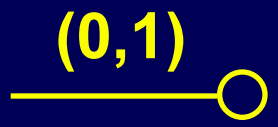


e.g.: Account (url, usr, pwd)
Address (in case of multiple addresses)

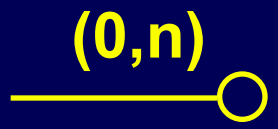
Optionality

Optional attributes:

("non-existence" of values is acceptable)

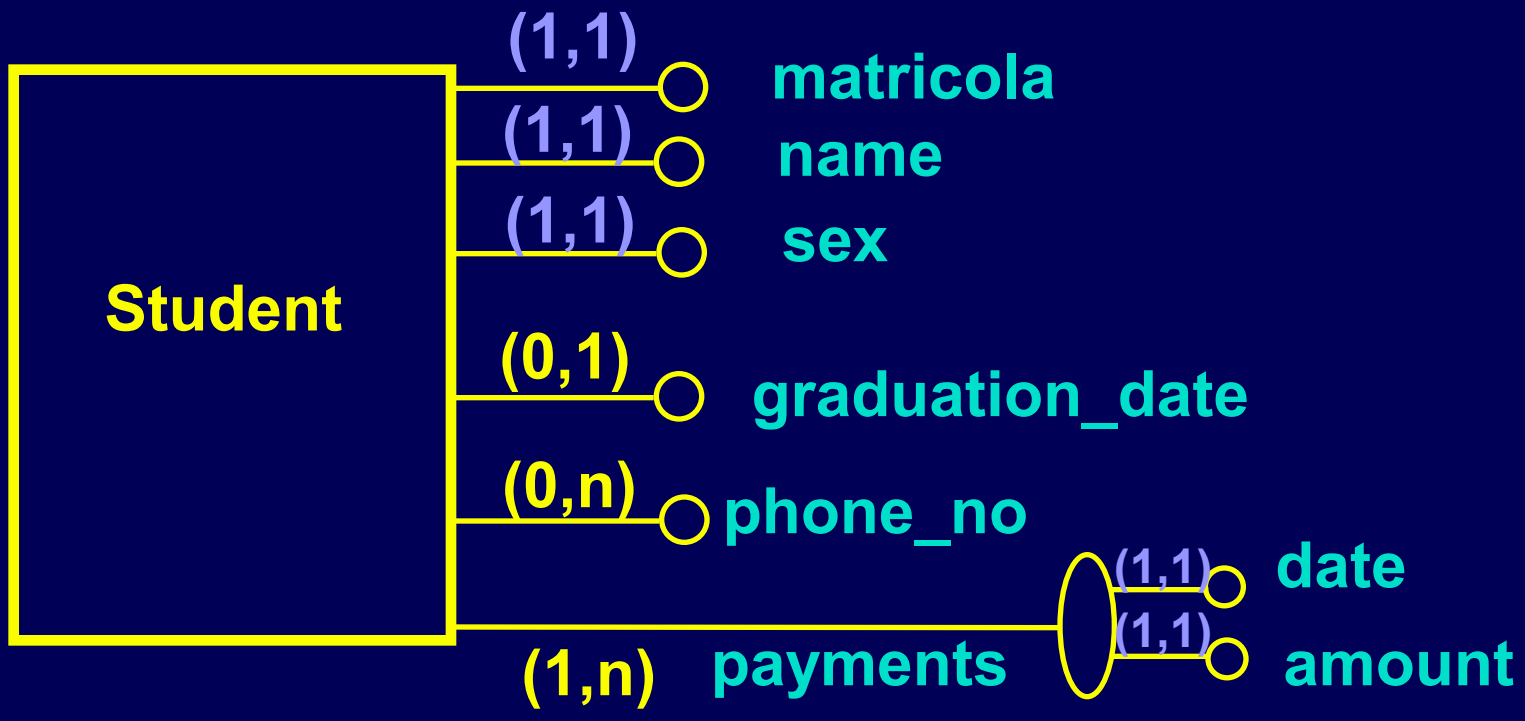


e.g.: DrivingLicence, …

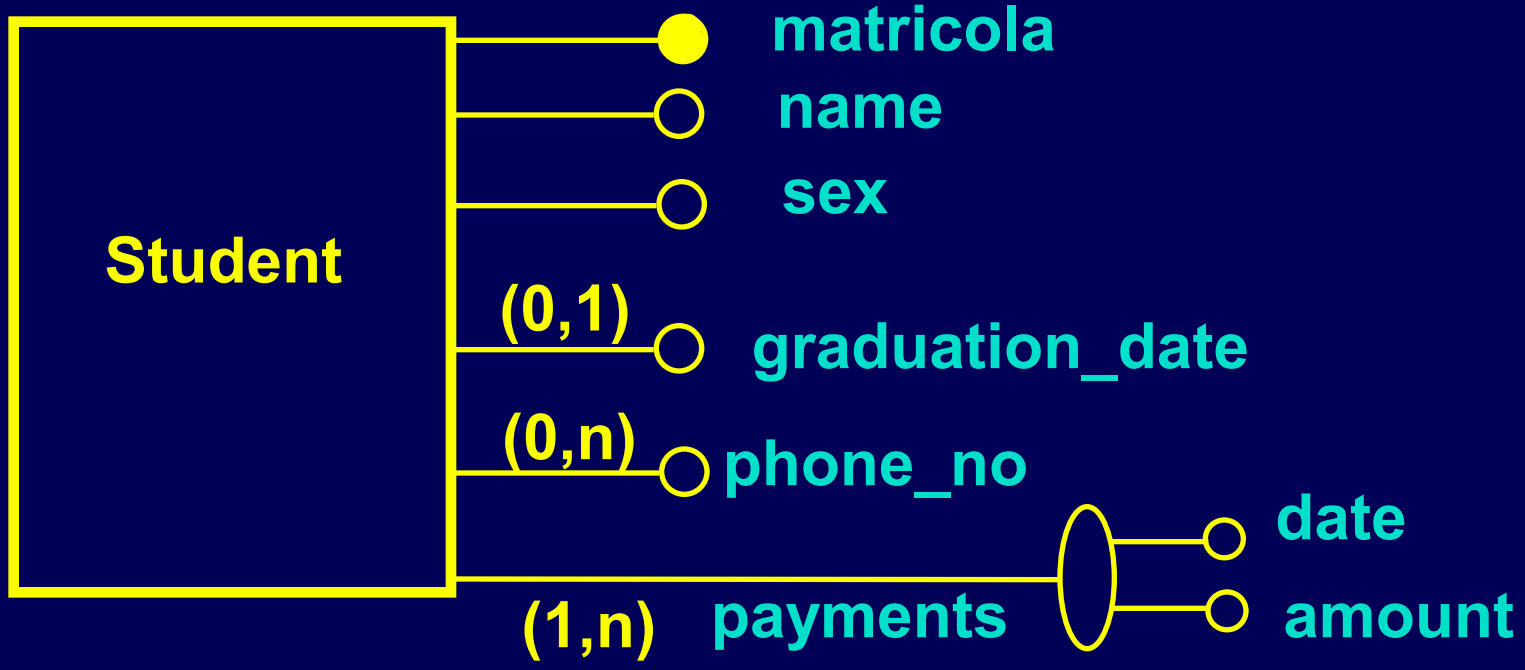


e.g.: PhoneNo, Diploma, …

Example


Student
(1,1)
matricola
(1,1)
name
(1,1)
sex
(0,1)
graduation_date
(0,n)
phone_no
(1,n)
payments
(1,1)
date
(1,1)
amount

Example with **default** cardinality (1,1)



(1,1) is conventionally assumed as the default only with attributes, not for associations, where cardinalities should always be explicit

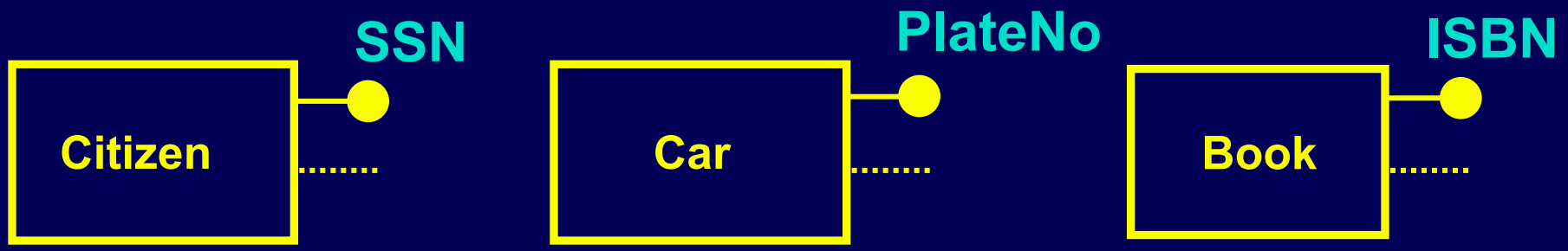
Identifiers

An **identifier** uniquely characterizes each single instance of an entity

- similar to a key at the logical level

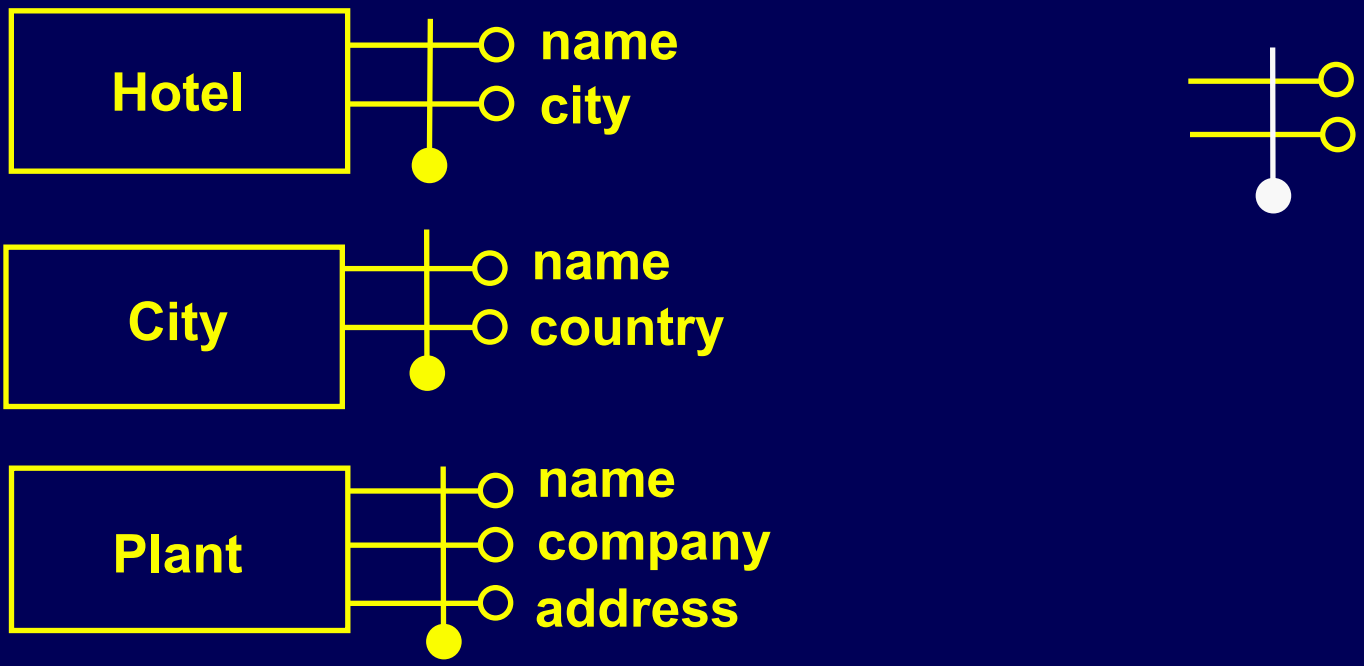
symbol: —●

Typically not updatable (or not frequently updated)



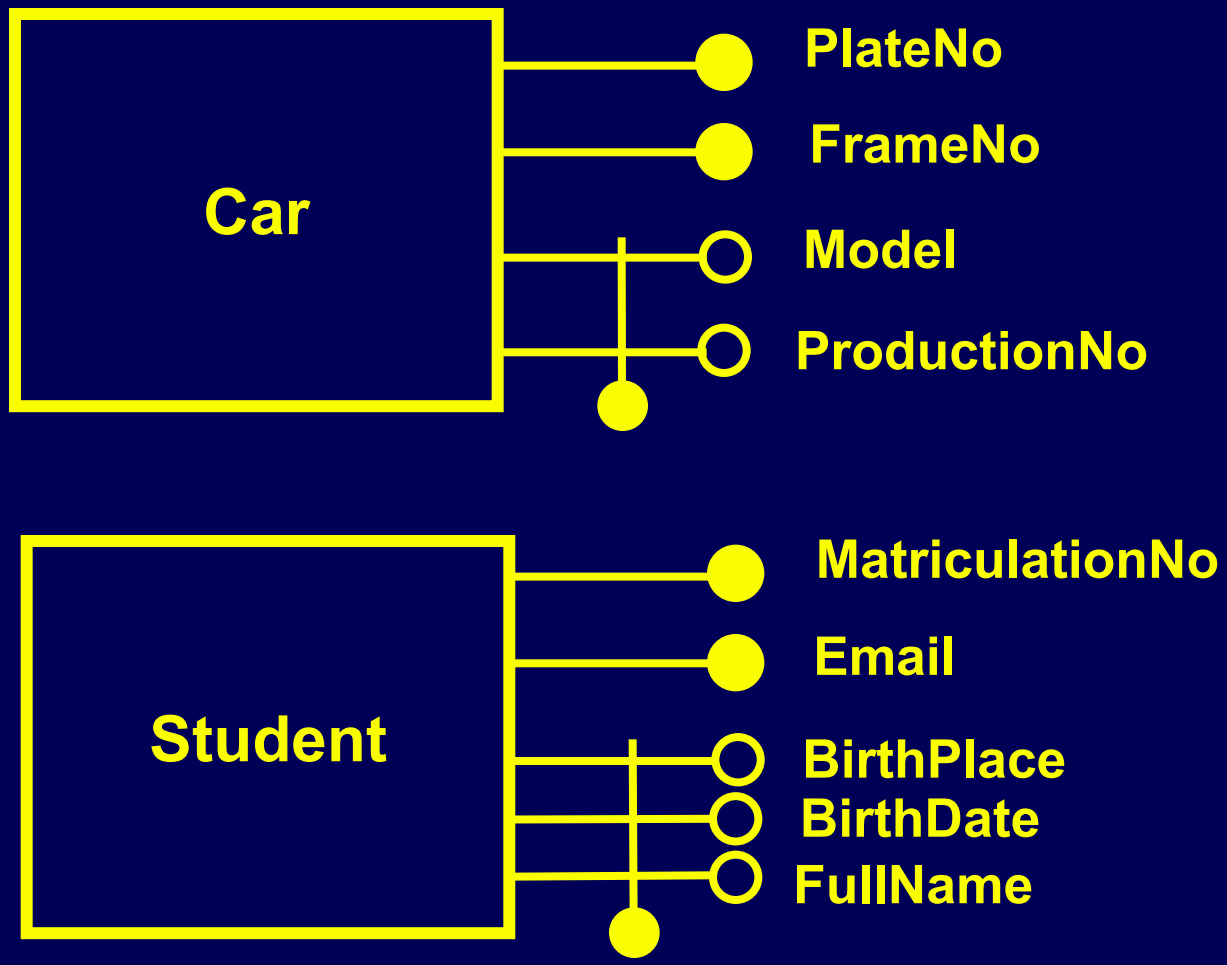
Composite Identifiers

Like other atributes, identifiers can be **composite**



A composite attribute is a good identifier if it is minimal
(once more, just like keys at the logical level)

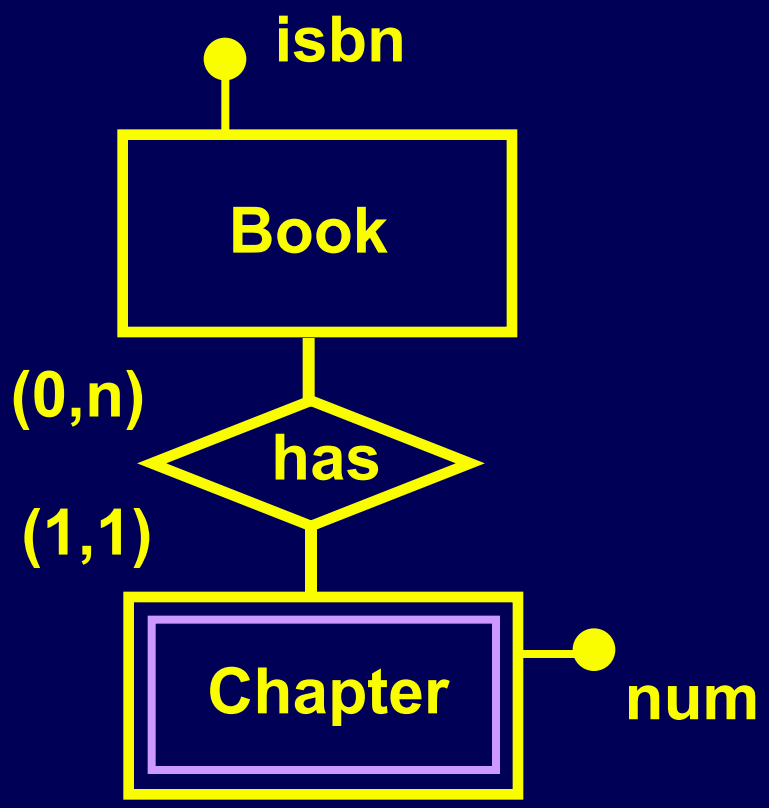
Entities may have more than one identifier

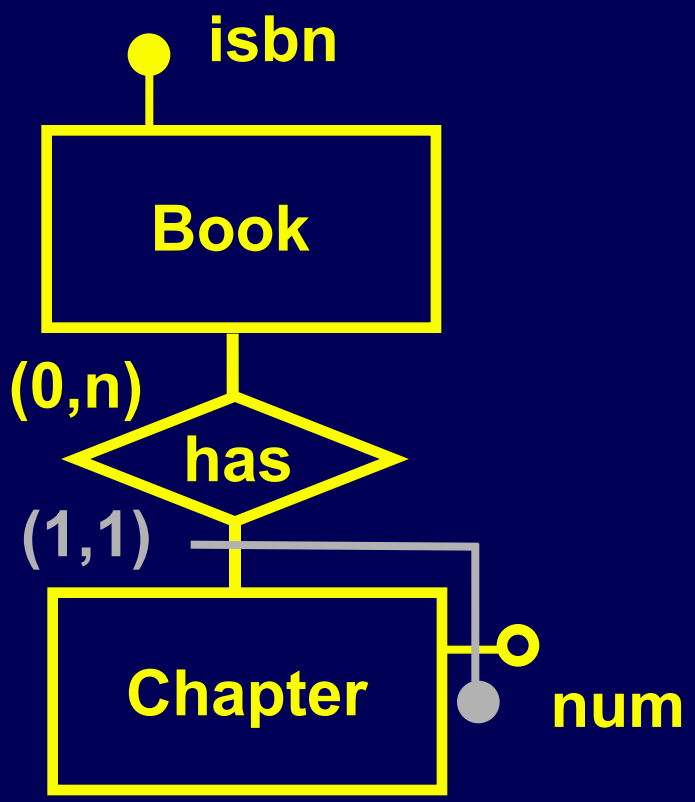


Weak entities

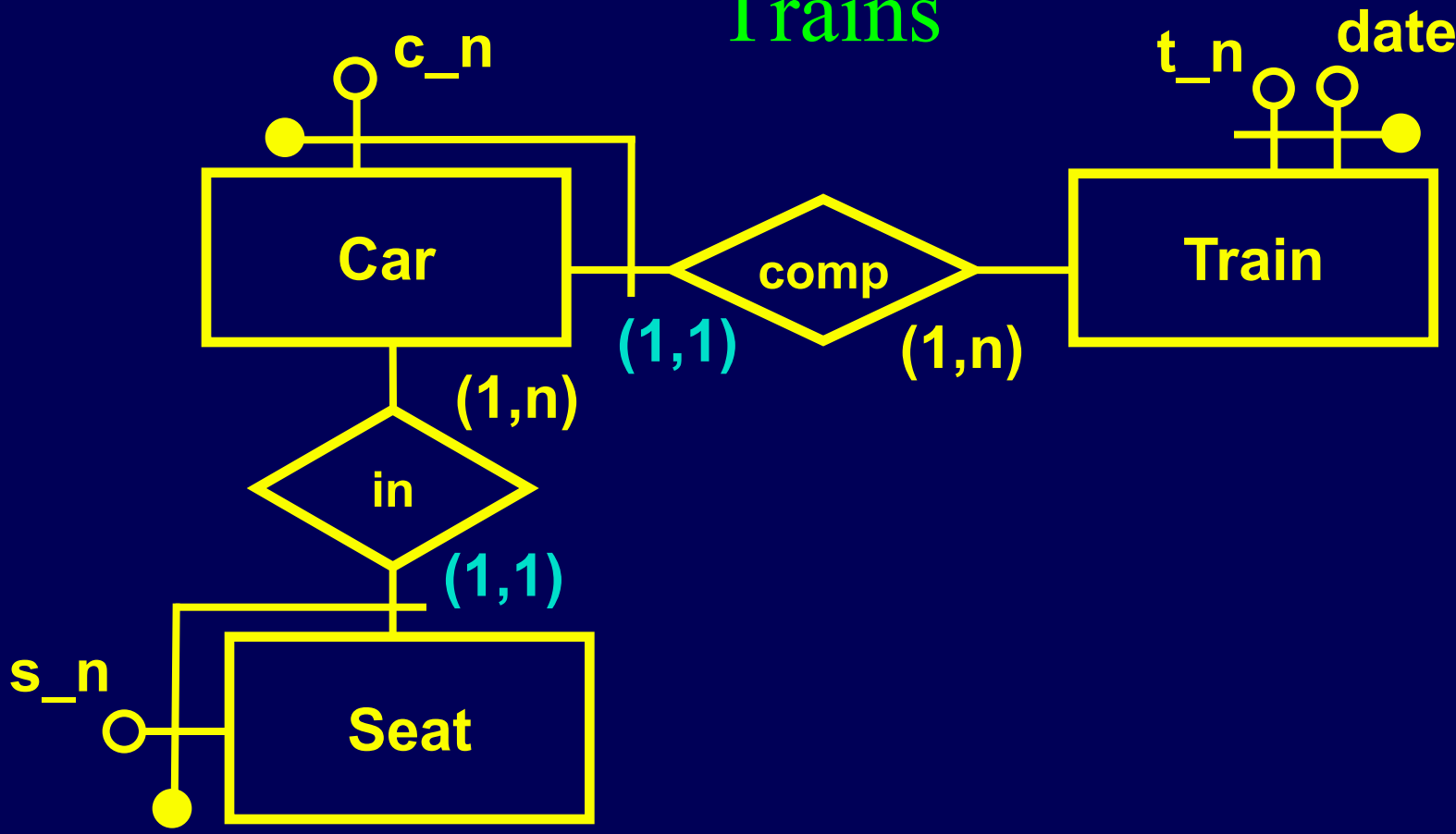
- Weak entities model objects and facts that may exist in the application context (i.e., make sense) only if other «strong» entities are present too (and they are dependent of those stronger entities)
  - The identifier of the stronger entity is required in order to identify the weak one

Symbols (the notation is not unique in literature)

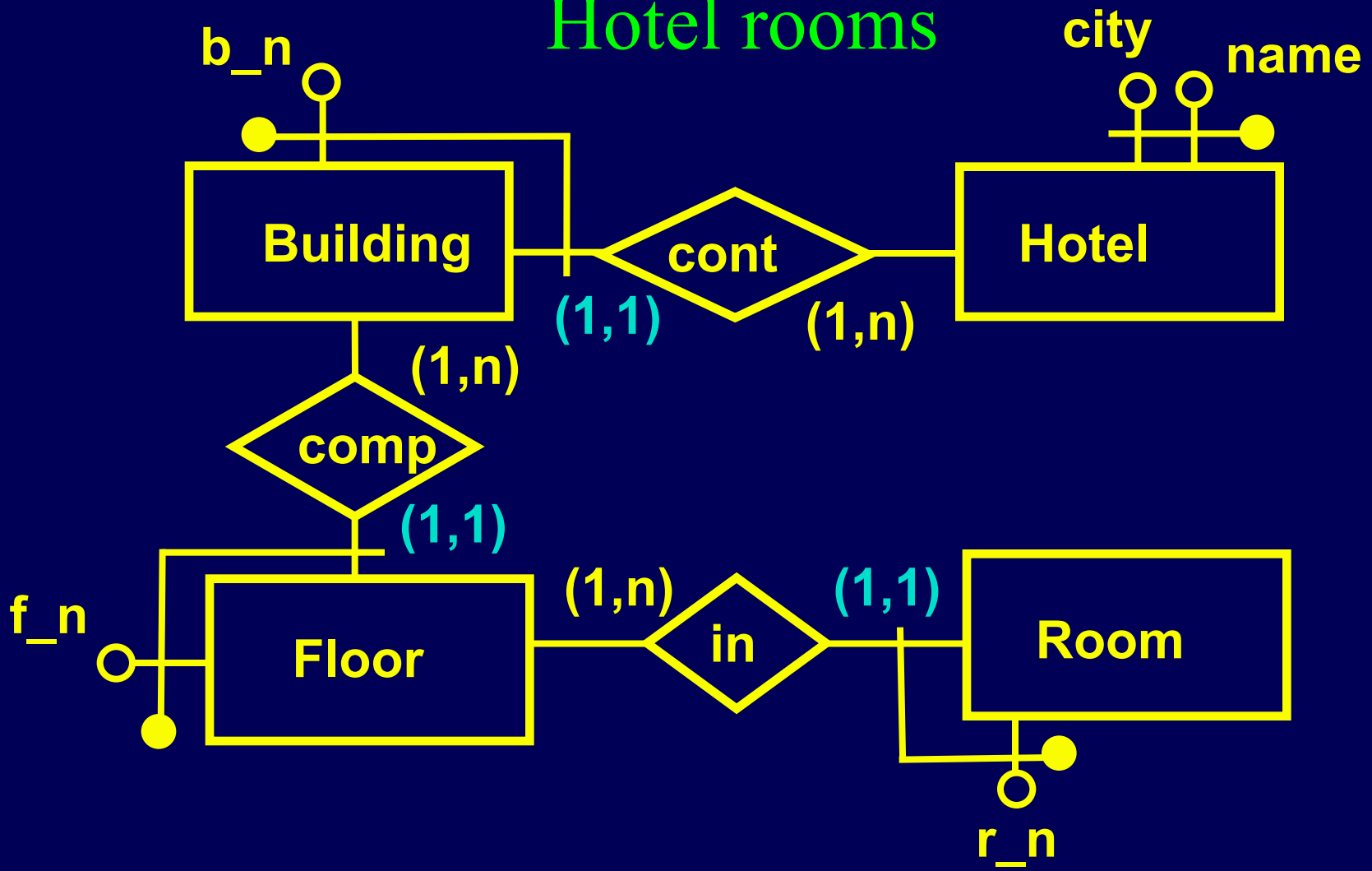




Trains



Hotel rooms


b_n
city
name
Building
cont
Hotel
(1,1)
(1,n)
(1,n)
comp
(1,1)
f_n
Floor
(1,n)
in
(1,1)
Room
r_n

A remark on «weakness»

- Entities with external identification are *weak* because a deletion of an (instance of an) entity at any level causes the deletion of all the weak entities that are associated to the deleted item

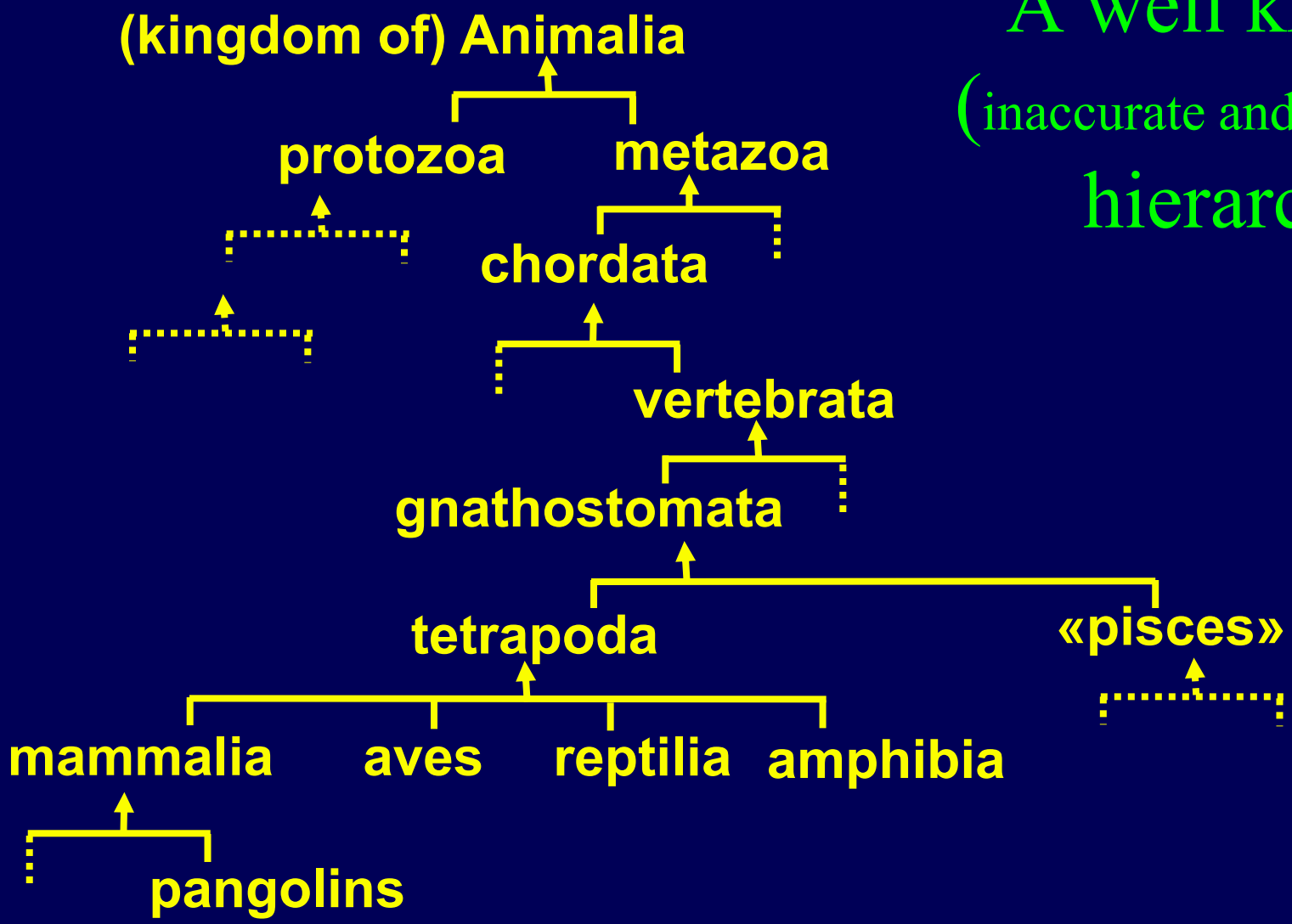
  - Removal of a train car → seats
  - Closure of a hotel floor → rooms

Generalization Hierarchies

Generalization hierarchies

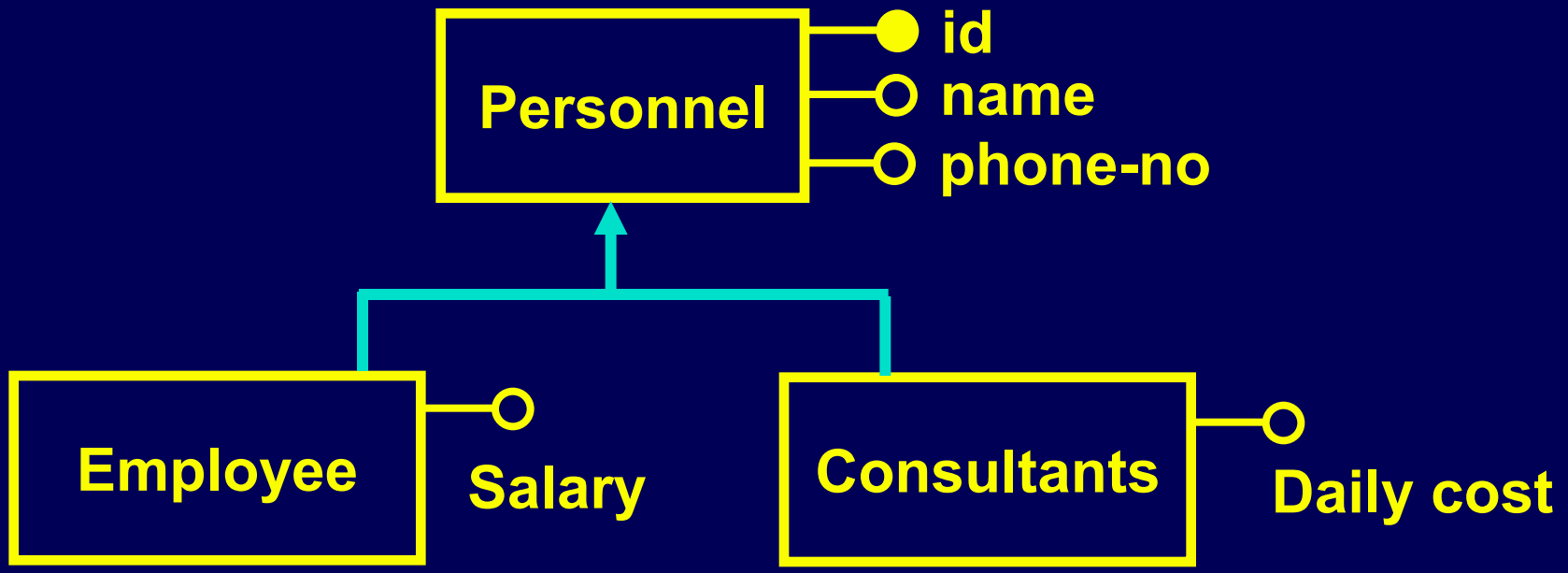
- A ***Generalization hierarchy*** is a binding between a «father» entity E and some «child» entities E_1 ... E_n such that, for each E_i:
  - every instance of E_i **is also** an instance of E
  - every instance of E **can be** an instance of E_i
- E is a ***generalization*** of E_1 E_2 ... E_n
- E_1 E_2 ... E_n are ***specializations*** of E
- «Child» entities ***inherit*** all the properties of their «fathers»
  - Attributes, relationships, identifiers

A well known (inaccurate and obsolete) hierarchy



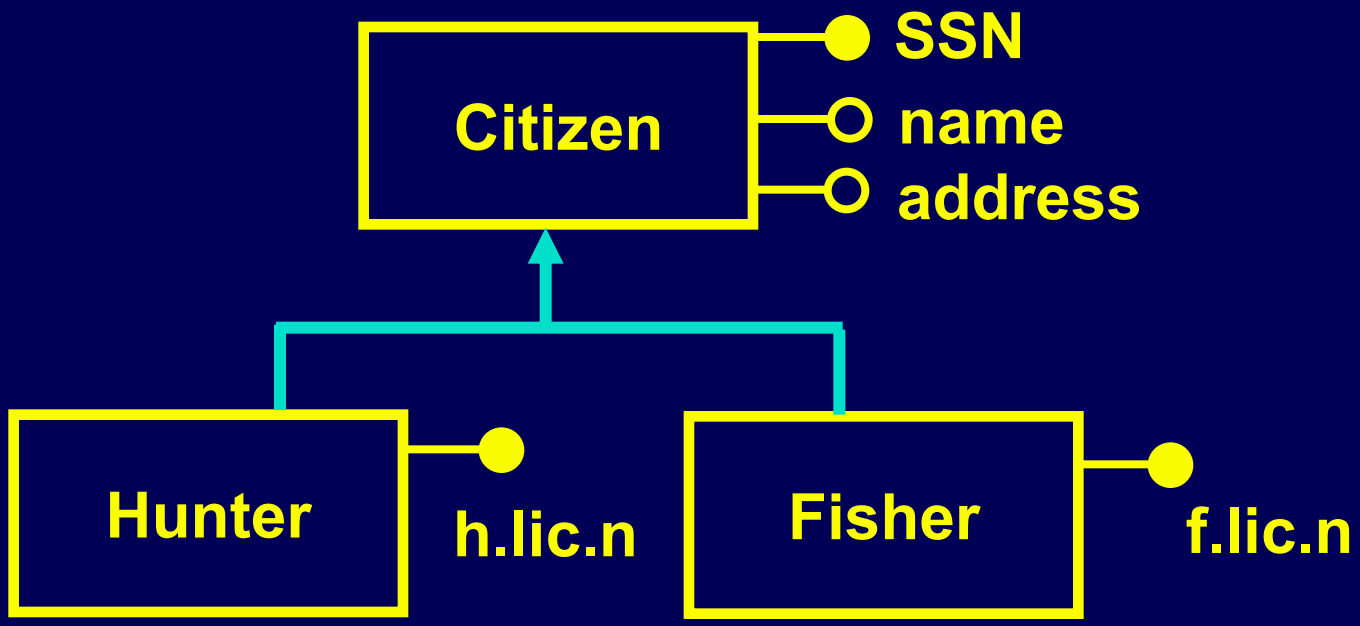
Other examples

A company can hire external professional consultants.
The personnel is thus made of employees and consultants:



Other examples

A municipality registers resident citizens, who may apply for a fishing or hunting licence:



Properties of hierarchies: *t* vs *p*

- A hierarchy is **total** (***t***) if each instance of the father entity **must** be an instance of **at least one** child entity
  - Personnel: a member is either an employee or a consultant

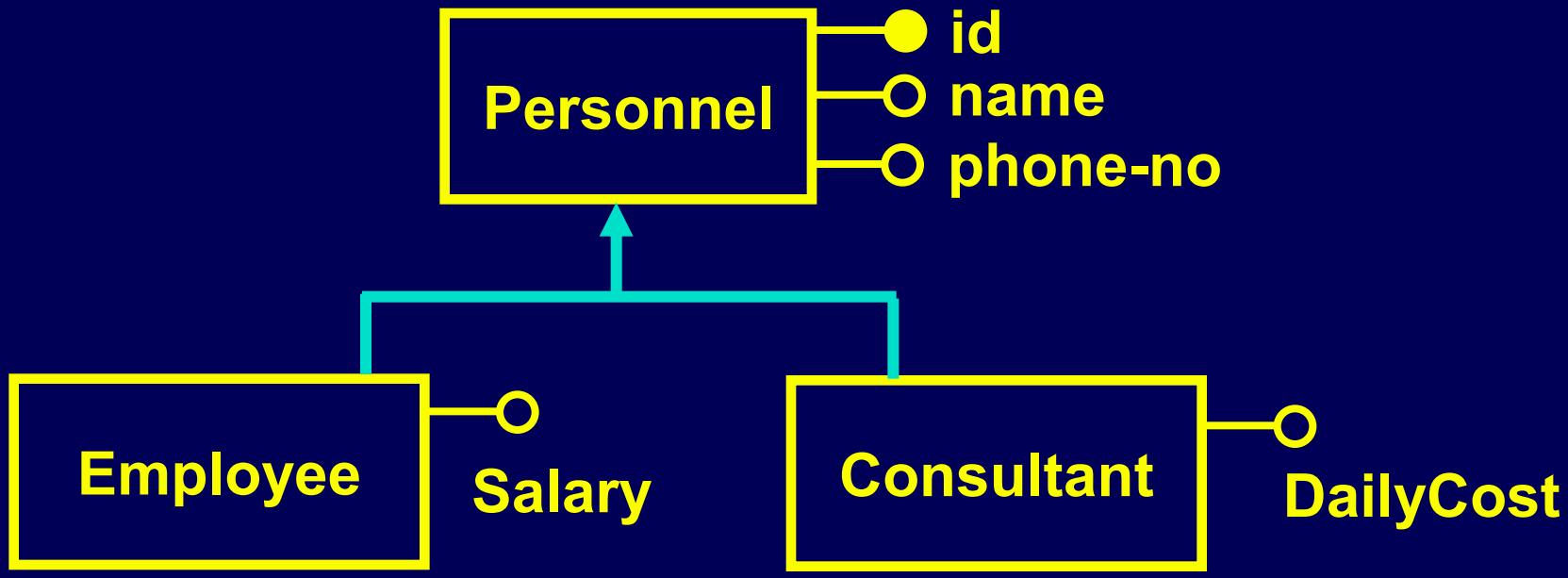
- A hierarchy is **partial** (***p***) if the instances of the father entity **may** be instances of the child entities or not
  - Clearly there are citizens who are neither fishers nor hunters

Properties of hierarchies: *e* vs *o*

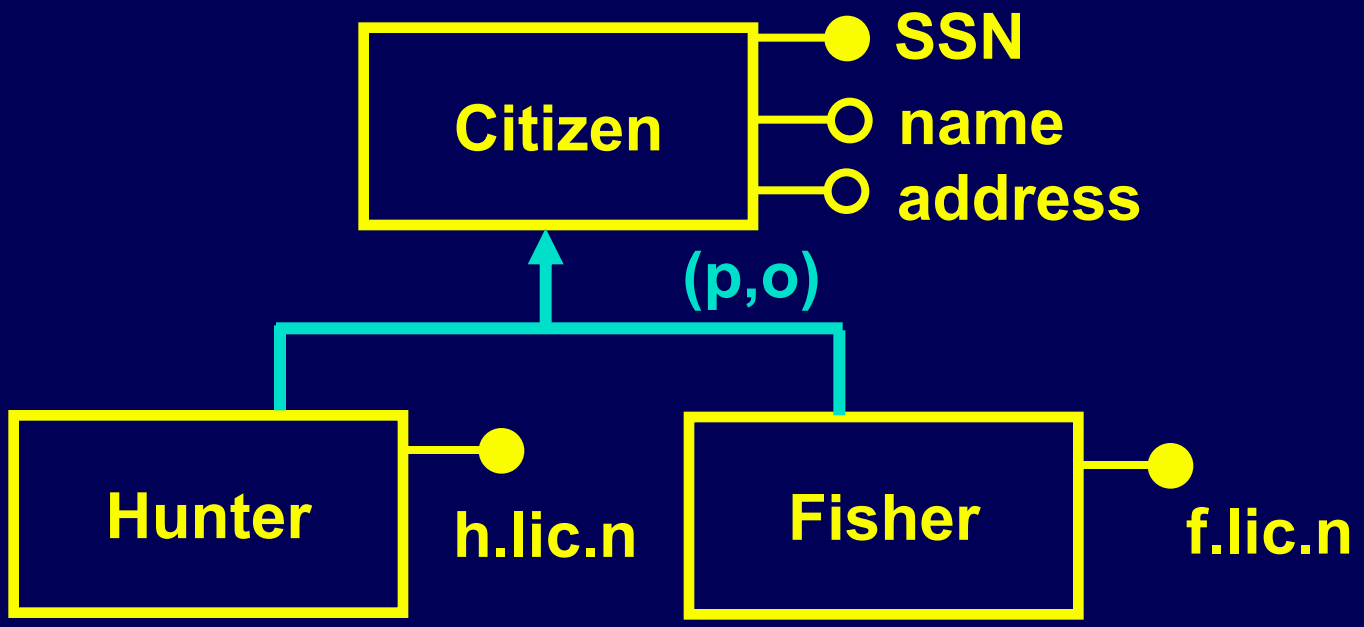
- A hierarchy is **exclusive** (**e**) if each instance of the father entity **cannot** be an instance of **more than one** child entity
  - Personnel: a member cannot be an employee ***and*** a consultant

- A hierarchy is **overlapping** (**o**) if the instances of the father entity **can** be instances of **multiple** child entities
  - Clearly a citizen can be a fisher ***and also*** a hunter

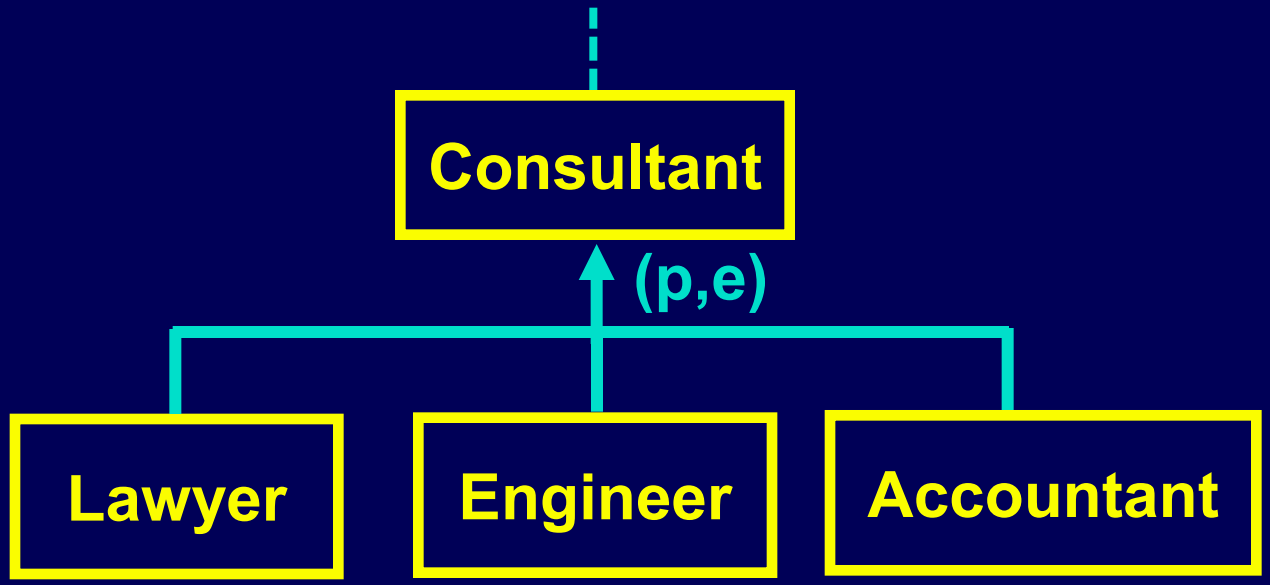
Default properties: (t,e)



With explicit properties



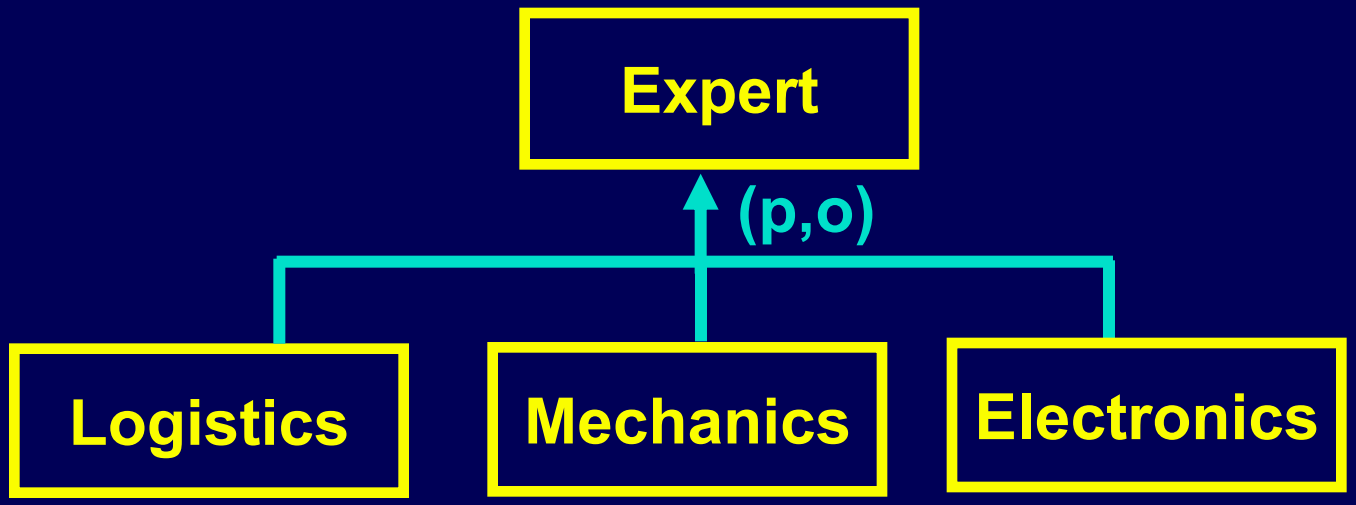
Further specializations



p: consultants can also have other qualifications, but there is no reason to define an ad-hoc fourth sub-class for the «other types»

e: the case of a lawyer who happens to also be an engineer is unlikely, but most importantly it is *not relevant for our application domain* (although not strictly impossible)

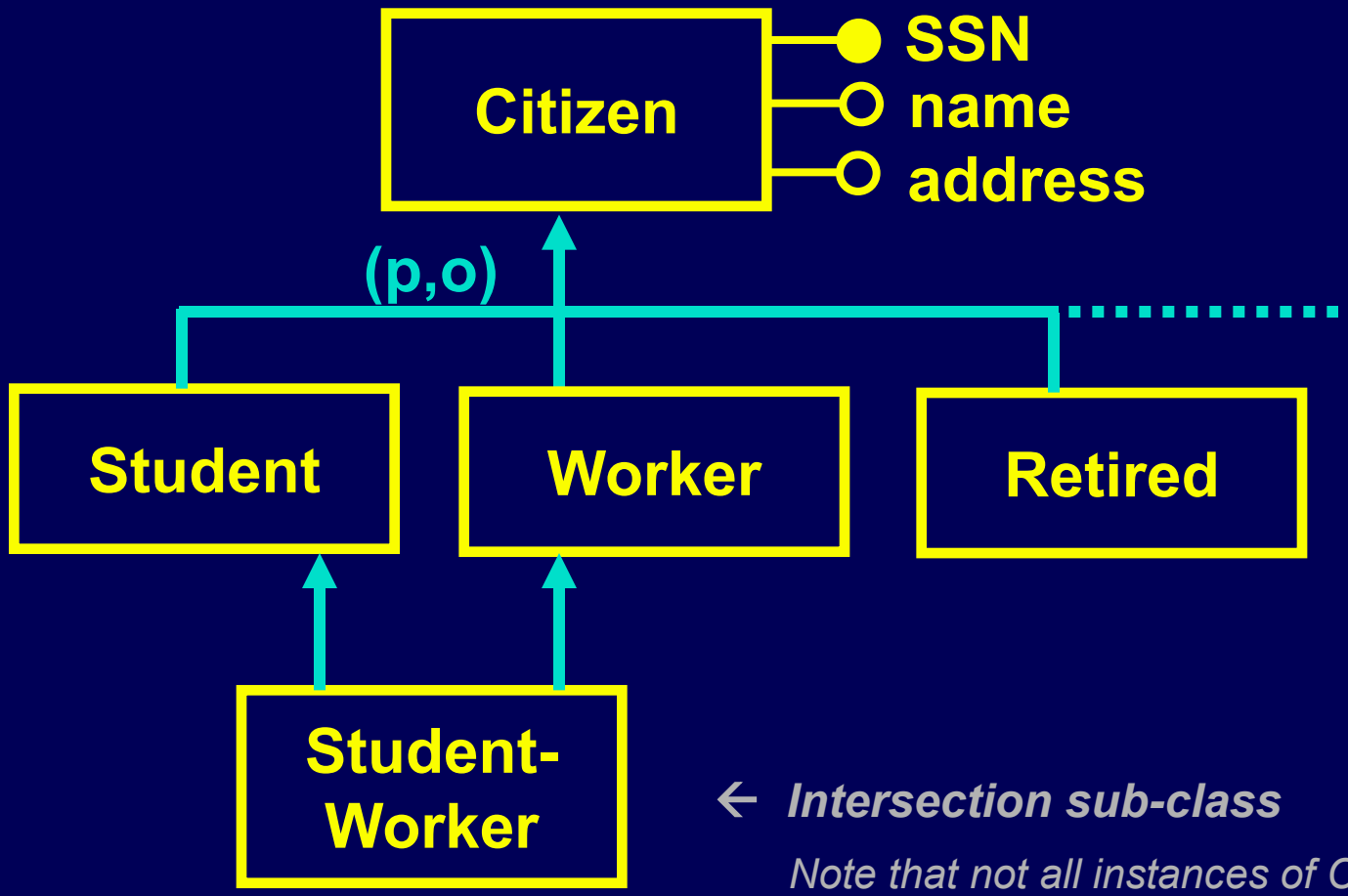
Further specializations



p: there may also be some expert in **other disciplines**

o: the three disciplines, however, are **not mutually exclusive**, as an expert can be qualified in more than one field

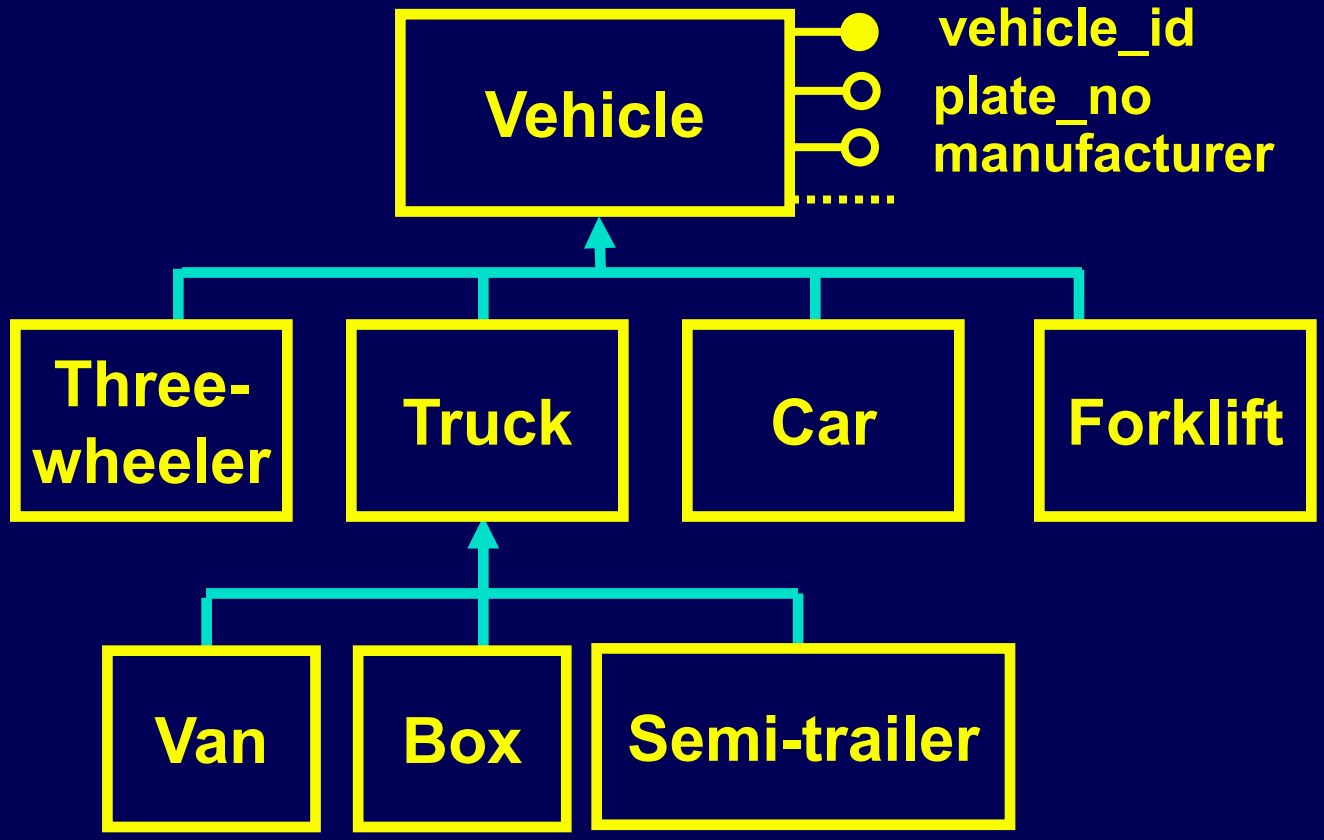
Multiple inheritance



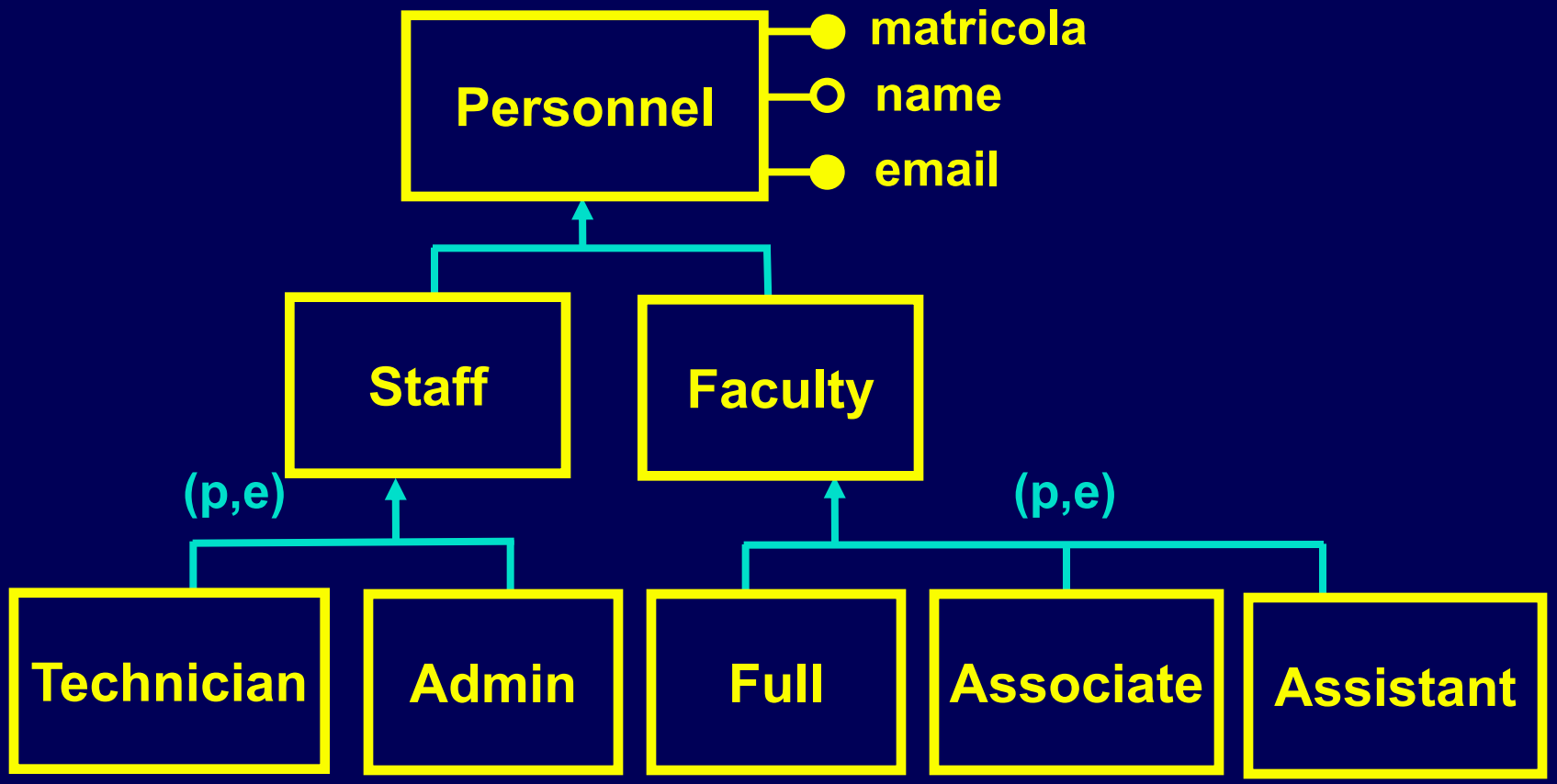
← ***Intersection sub-class***

Note that not all instances of Citizen who are Students and Workers must necessarily be considered part of the intersection. The class may also represent a property that requires some «voluntary» subscription to a category.

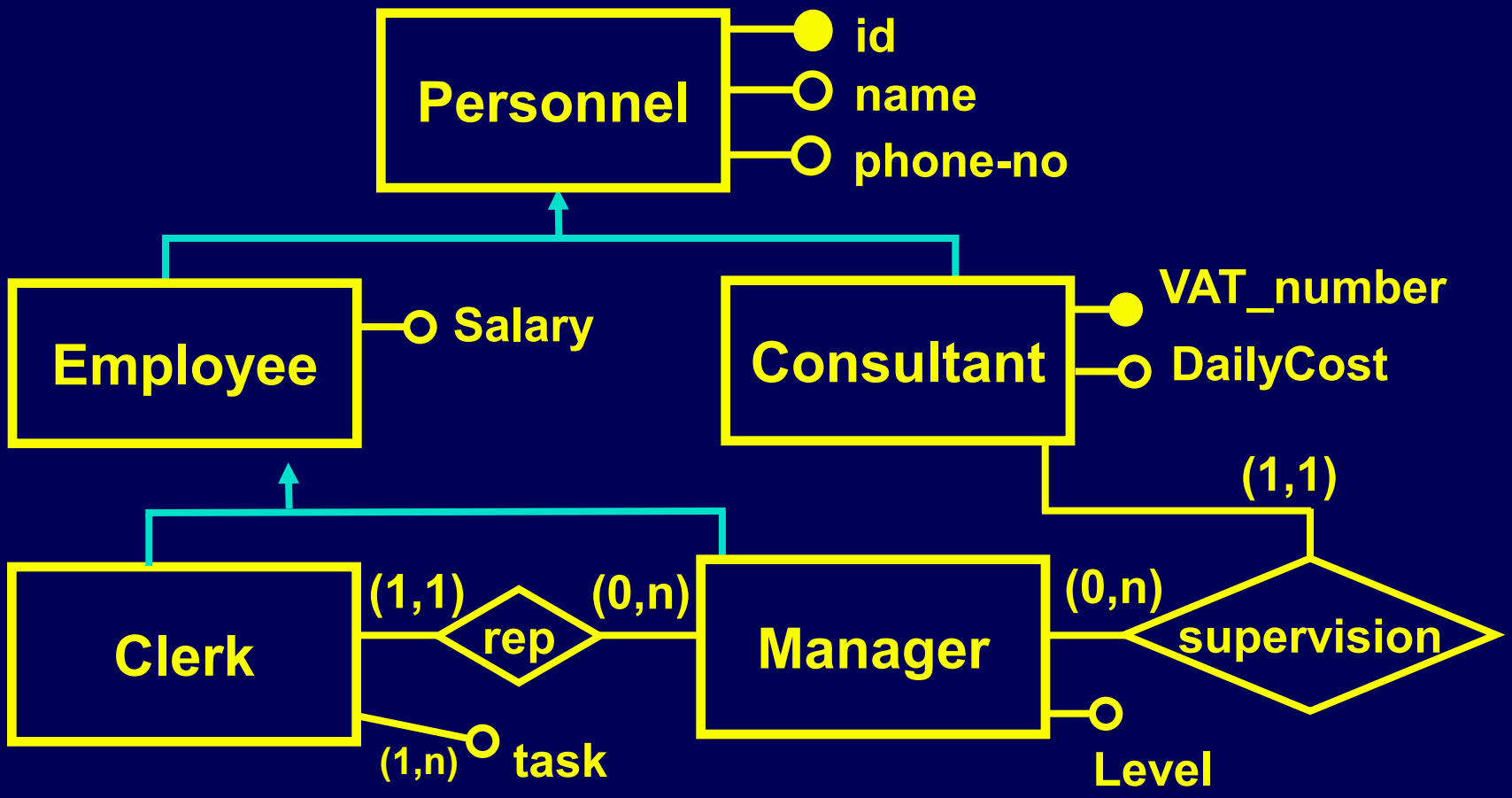
Industrial vehicles

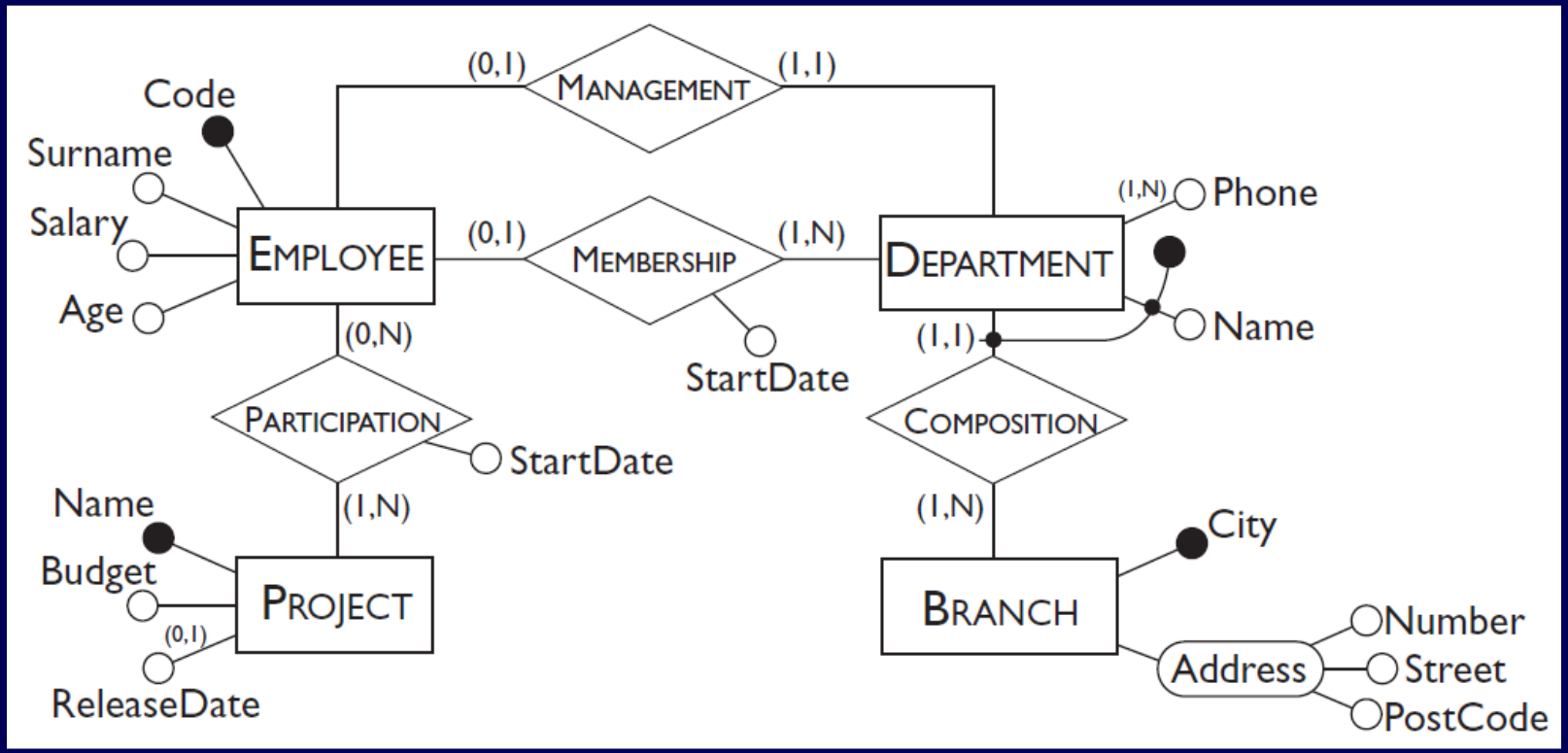


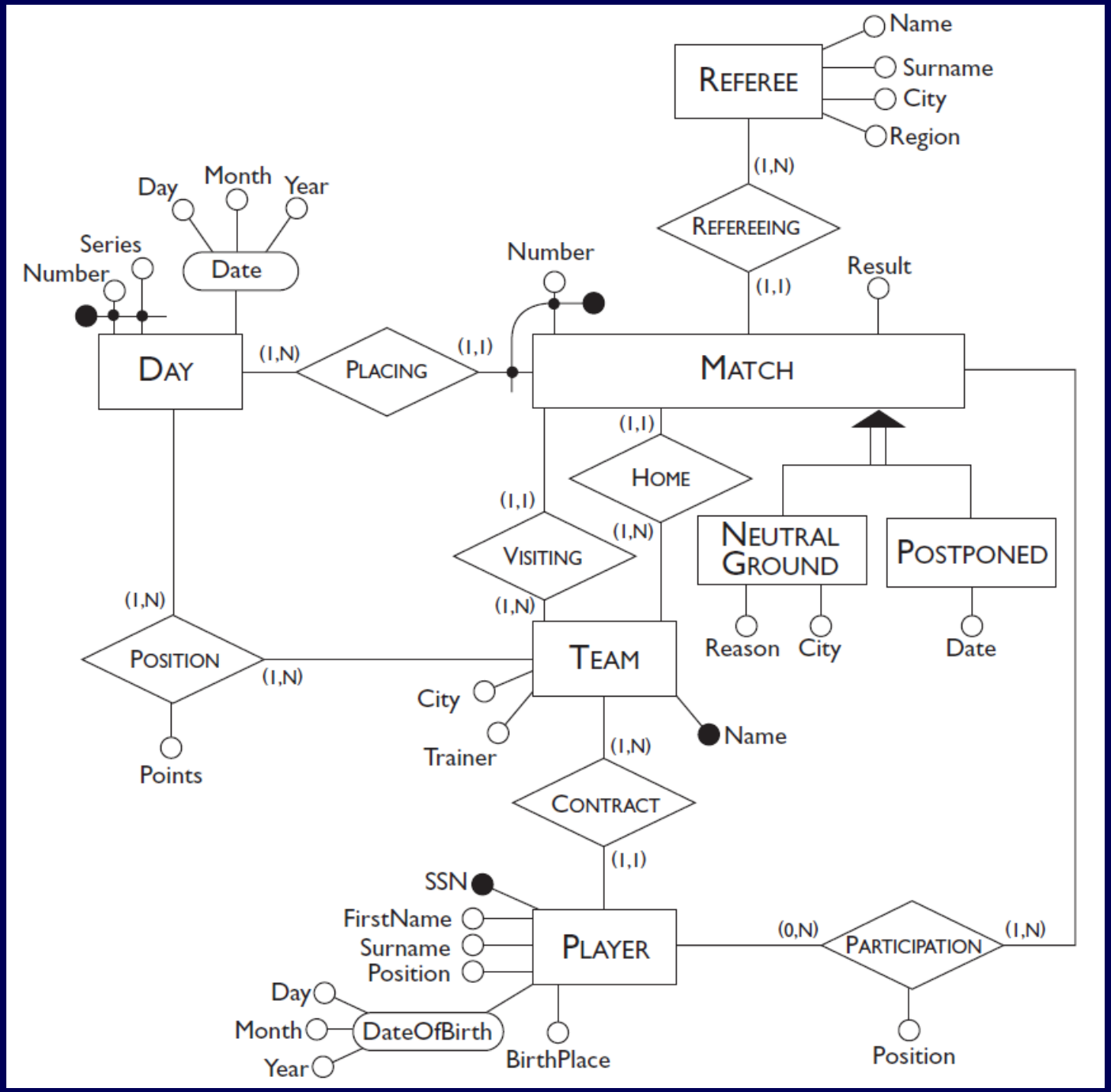
Academic Personnel

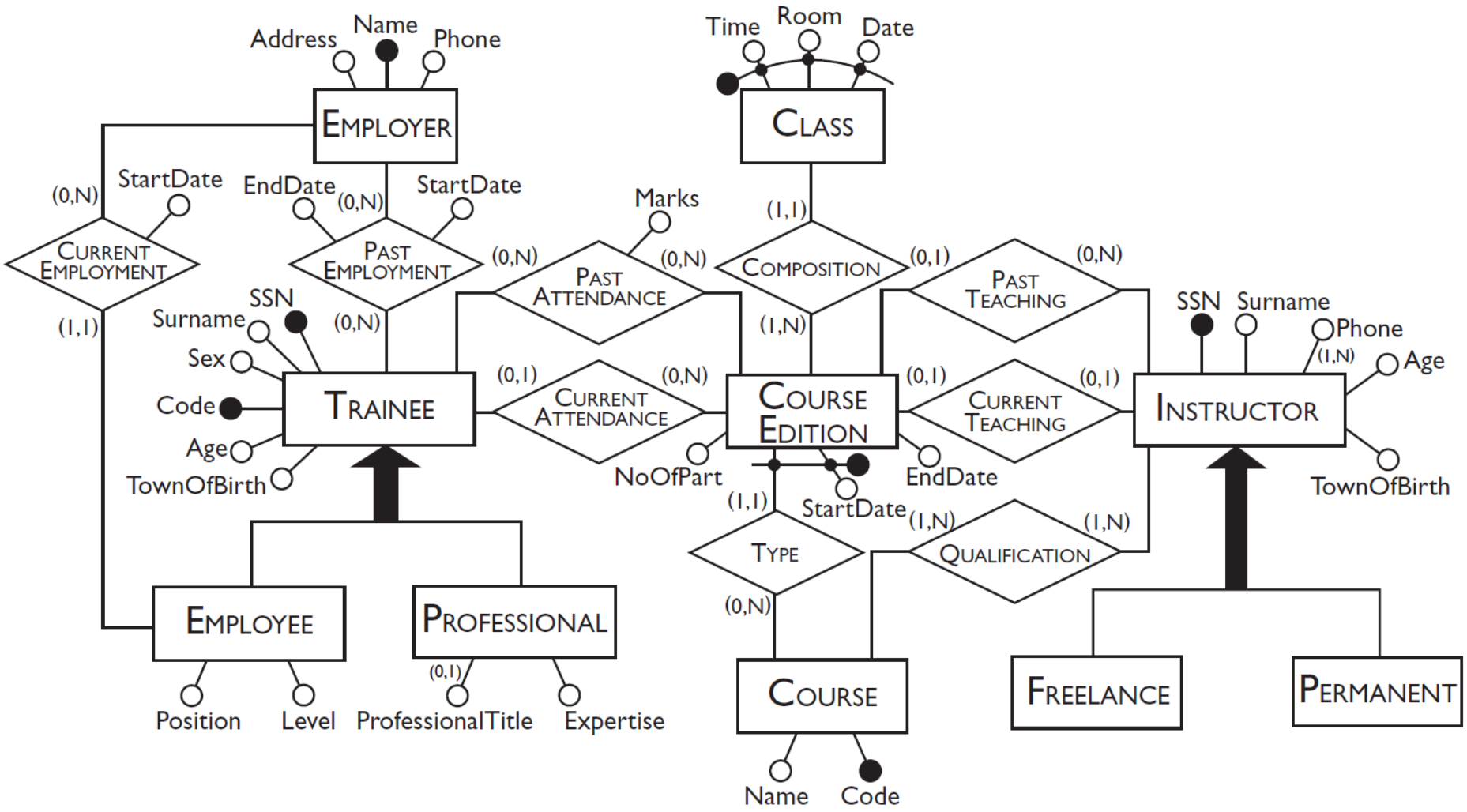


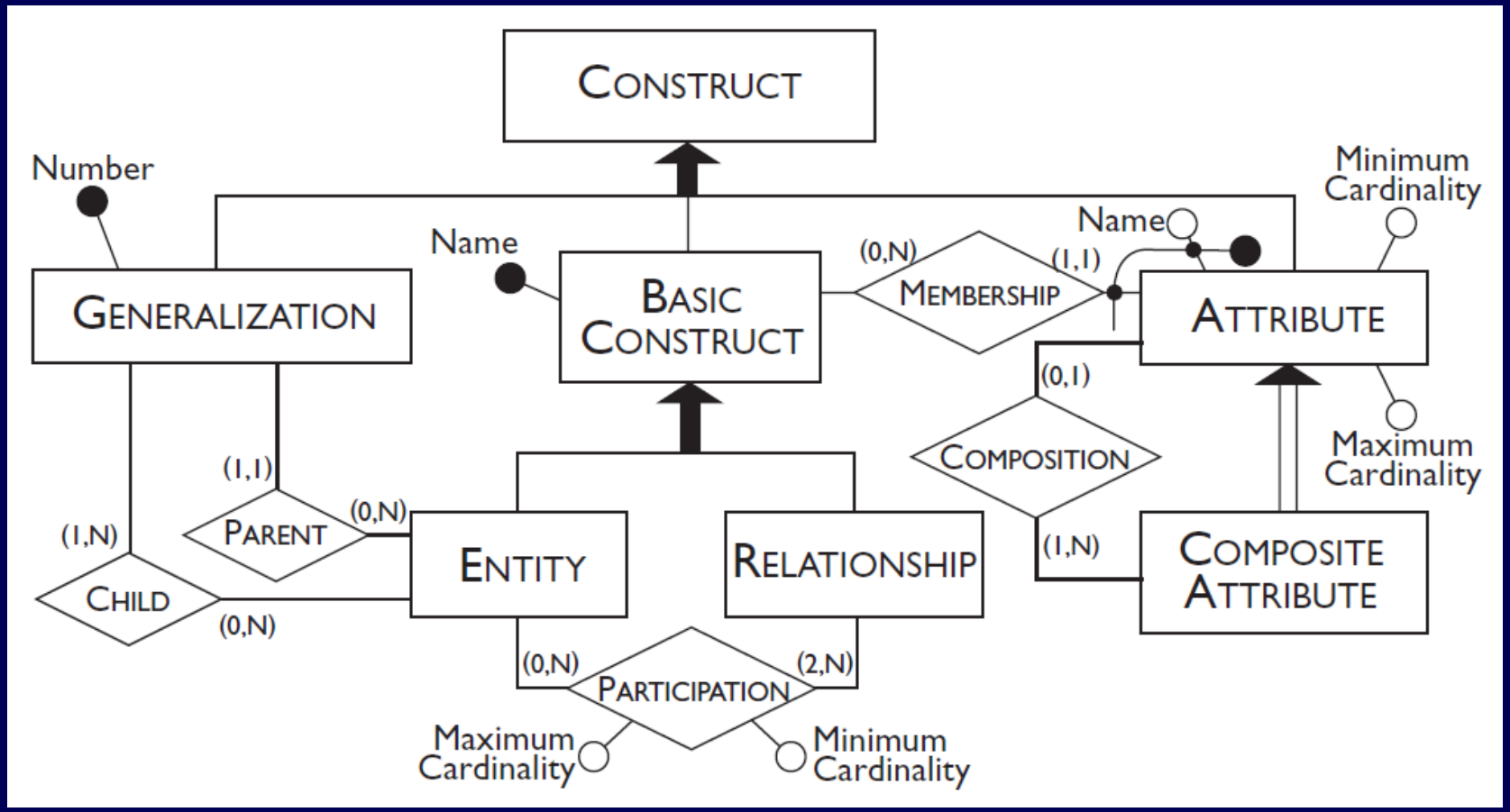
Company personnel

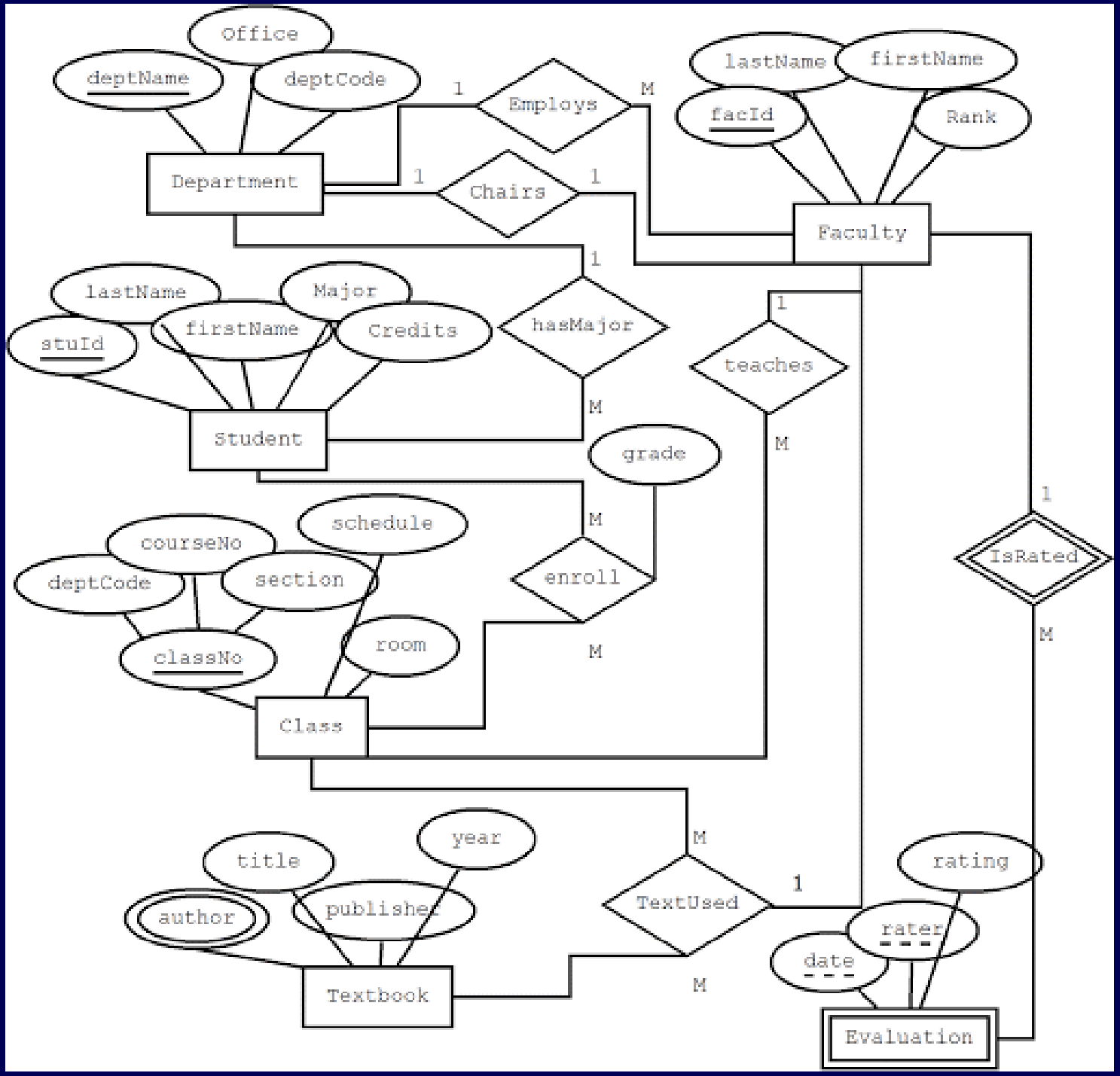


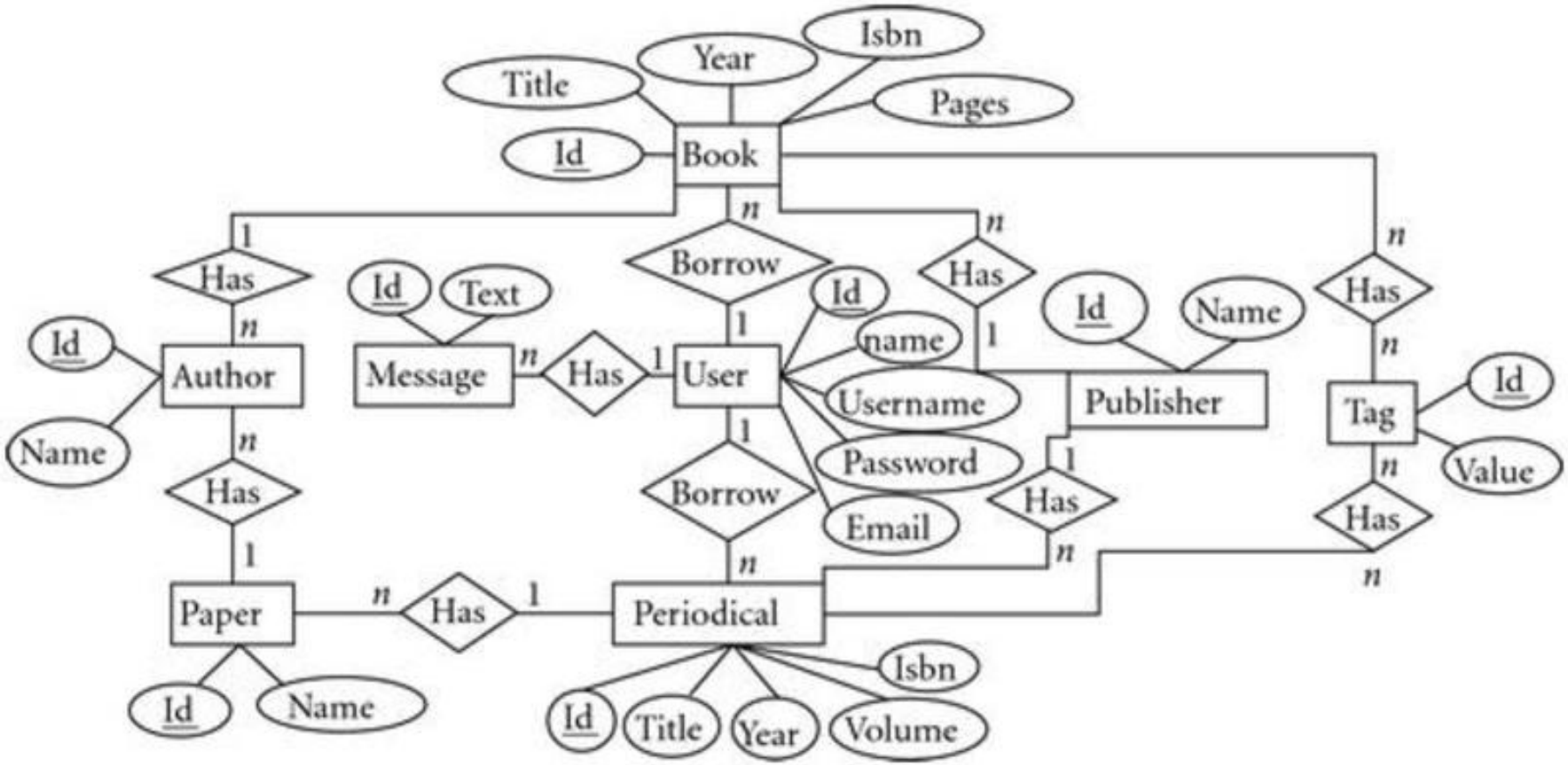

(0,1)
MANAGEMENT
(1,1)
Code
Surname
Salary
Age
EMPLOYEE
(0,1)
MEMBERSHIP
(1,N)
DEPARTMENT
(1,N)
Phone
Name
StartDate
(0,N)
(1,1)
PARTICIPATION
StartDate
COMPOSITION
Name
Budget
(0,1)
ReleaseDate
(1,N)
PROJECT
(1,N)
City
BRANCH
Address
Number
Street
PostCode

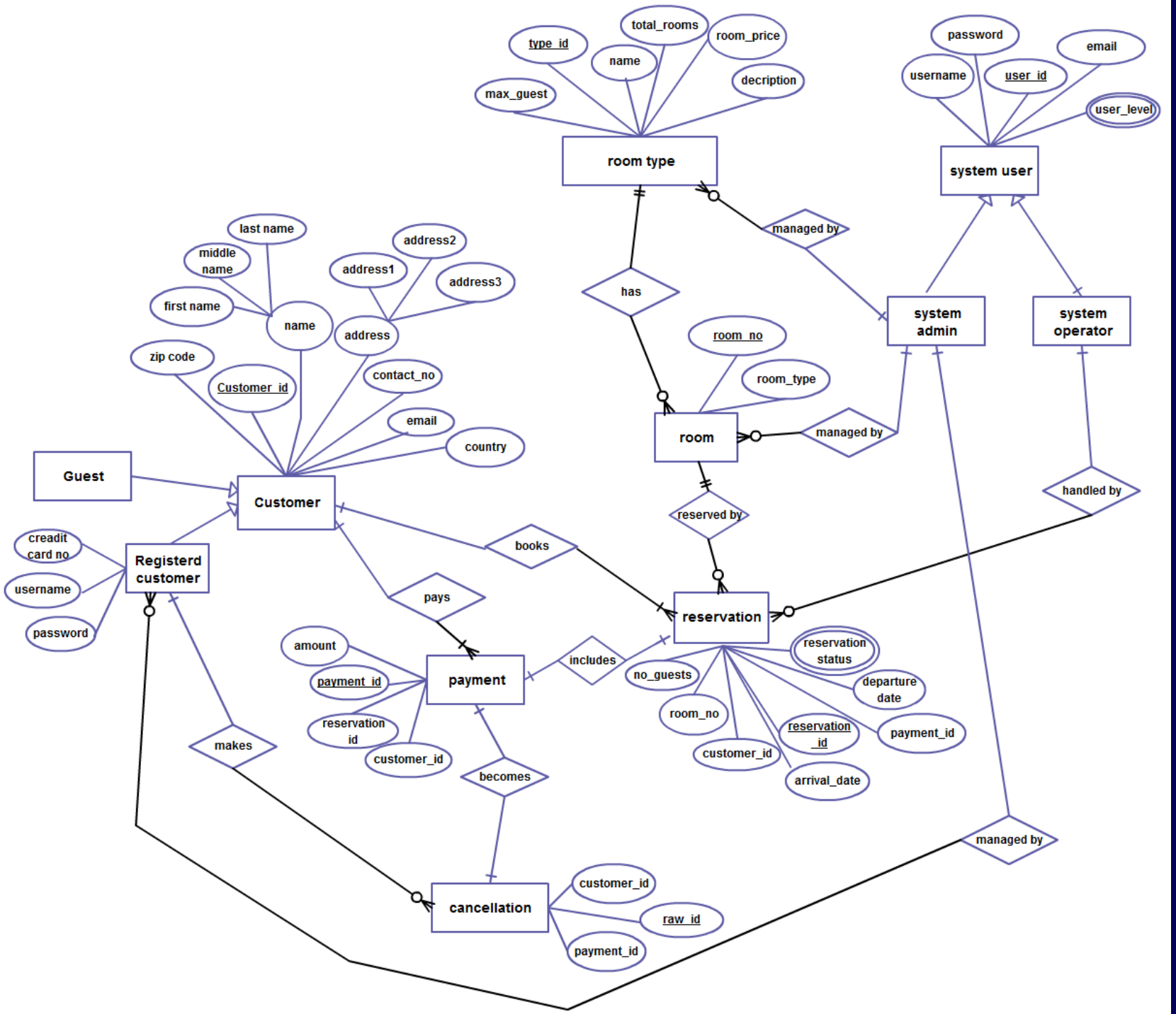

REFEREE
Name
Surname
City
Region
(1,N)
REFEREEING
(1,1)
Result
Number
Day
Month
Year
Series
Number
Date
DAY
(1,N)
PLACING
(1,1)
MATCH
(1,1)
HOME
(1,1)
(1,N)
VISITING
(1,N)
NEUTRAL GROUND
POSTPONED
Reason
City
Date
(1,N)
POSITION
(1,N)
TEAM
City
Trainer
Name
Points
(1,N)
CONTRACT
(1,1)
SSN
FirstName
Surname
Position
PLAYER
(0,N)
PARTICIPATION
(1,N)
Day
Month
Year
DateOfBirth
BirthPlace
Position

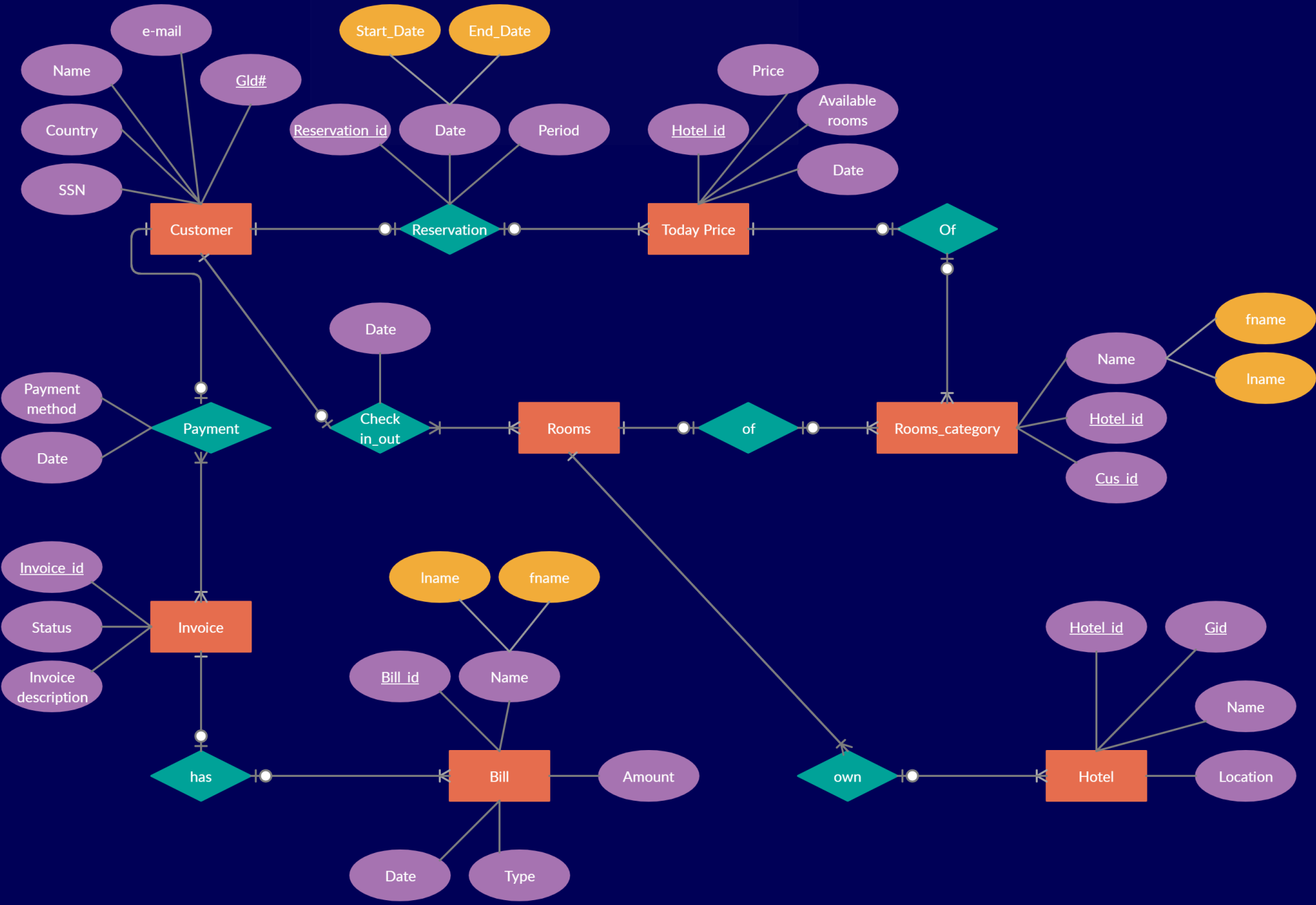

Address
Name
Phone
EMPLOYER
Time
Room
Date
CLASS
(0,N)
StartDate
CURRENT EMPLOYMENT
EndDate
(0,N)
StartDate
PAST EMPLOYMENT
Marks
(0,N)
PAST ATTENDANCE
(0,N)
(1,1)
COMPOSITION
(0,1)
(0,N)
PAST TEACHING
(1,1)
SSN
Surname
(0,N)
Sex
Code
Age
TownOfBirth
TRAINEE
(0,1)
(0,N)
CURRENT ATTENDANCE
(1,N)
COURSE EDITION
(0,1)
(0,1)
CURRENT TEACHING
SSN
Surname
Phone
(1,N)
Age
INSTRUCTOR
TownOfBirth
NoOfPart
EndDate
(1,1)
StartDate
(1,N)
(1,N)
TYPE
QUALIFICATION
(0,N)
EMPLOYEE
PROFESSIONAL
(0,1)
Position
Level
ProfessionalTitle
Expertise
COURSE
FREELANCE
PERMANENT
Name
Code

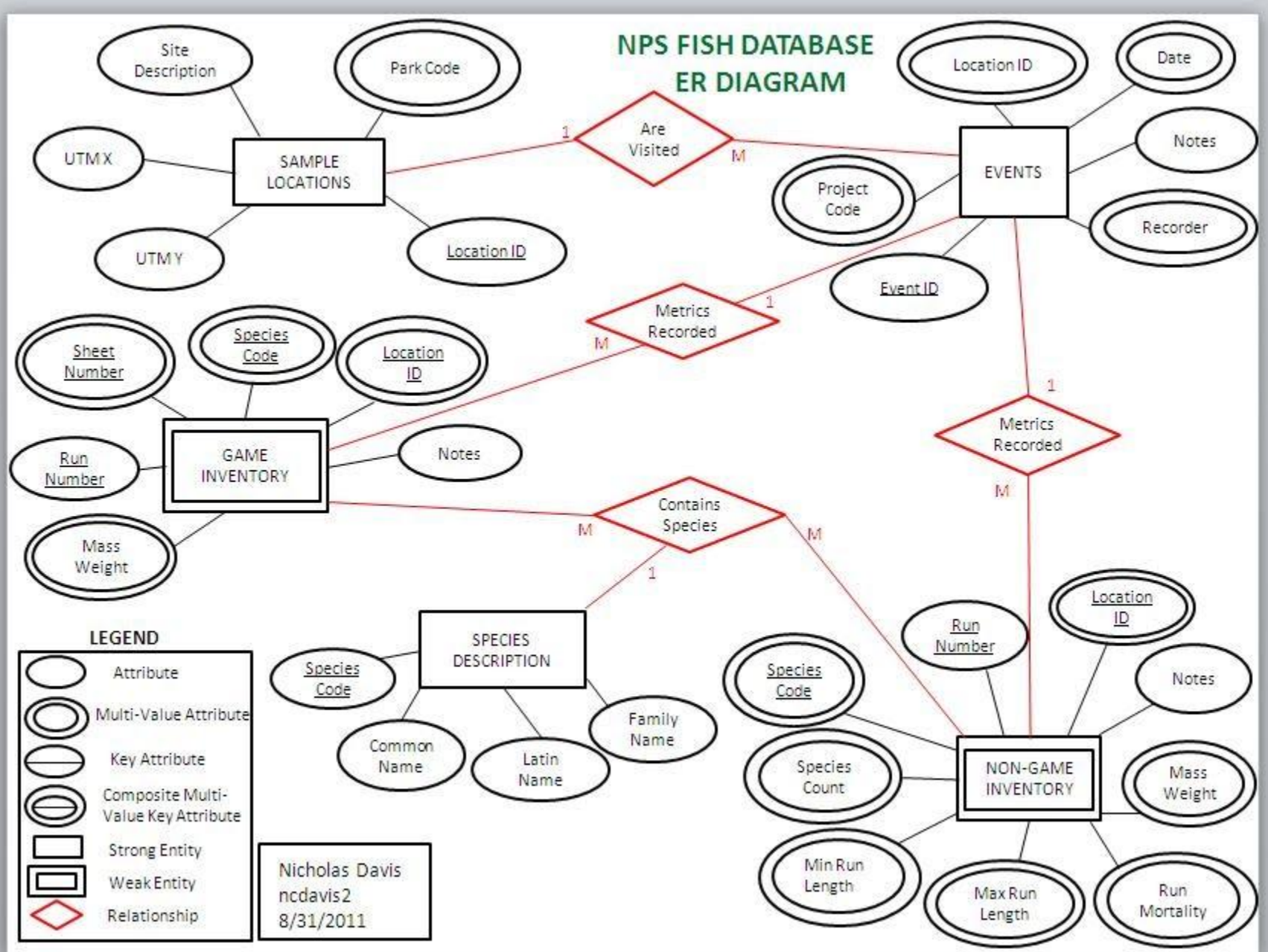

CONSTRUCT
Number
Name
Minimum Cardinality
Name
GENERALIZATION
BASIC CONSTRUCT
(0,N)
MEMBERSHIP
(1,1)
ATTRIBUTE
(0,1)
COMPOSITION
Maximum Cardinality
(1,1)
PARENT
(0,N)
(1,N)
CHILD
(0,N)
ENTITY
RELATIONSHIP
(1,N)
COMPOSITE ATTRIBUTE
(0,N)
PARTICIPATION
(2,N)
Maximum Cardinality
Minimum Cardinality


Office
deptName
deptCode
Employs
1
M
lastName
firstName
facId
Rank
Department
Chairs
Faculty
lastName
Major
firstName
Credits
stuId
hasMajor
teaches
Student
grade
schedule
courseNo
deptCode
section
enroll
IsRated
room
classNo
Class
year
title
author
publisher
TextUsed
rating
rater
date
Textbook
Evaluation


Isbn
Year
Title
Pages
Id
Book
Has
Borrow
Has
Has
Id
Text
Id
Name
Id
Author
Message
Has
User
name
Id
Name
Publisher
Tag
Id
Username
Value
Name
Password
Has
Borrow
Has
Email
Paper
Has
Periodical
Isbn
Id
Name
Id
Title
Year
Volume


room type
type_id
total_rooms
room_price
name
decription
max_guest
system user
password
email
username
user_id
user_level
managed by
has
system admin
system operator
last name
middle name
address2
address1
address3
first name
name
address
zip code
Customer_id
contact_no
email
country
room_no
room_type
room
managed by
Guest
Customer
handled by
reserved by
creadit card no
Registerd customer
username
password
books
pays
reservation
reservation status
amount
payment_id
payment
includes
no_guests
departure date
room_no
reservation_id
payment_id
customer_id
arrival_date
reservation id
customer_id
makes
becomes
cancellation
customer_id
raw_id
payment_id
managed by


e-mail
Name
Gld#
Country
SSN
Customer
Start_Date
End_Date
Reservation_id
Date
Period
Reservation
Price
Available rooms
Hotel_id
Date
Today Price
Of
Date
Payment method
Payment
Date
Check in_out
Rooms
of
Rooms_category
Name
fname
lname
Hotel_id
Cus_id
Invoice_id
Status
Invoice description
Invoice
lname
fname
Bill_id
Name
has
Bill
Amount
Date
Type
own
Hotel_id
Gid
Name
Hotel
Location


NPS FISH DATABASE
ER DIAGRAM
Site Description
Park Code
UTM X
SAMPLE LOCATIONS
UTM Y
Location ID
1
Are Visited
M
Location ID
Date
Notes
EVENTS
Project Code
Recorder
Event ID
Metrics Recorded
1
M
Sheet Number
Species Code
Location ID
Run Number
GAME INVENTORY
Notes
Mass Weight
Contains Species
M
M
1
Metrics Recorded
1
M
Run Number
Location ID
Species Code
Notes
NON-GAME INVENTORY
Species Count
Mass Weight
Min Run Length
Max Run Length
Run Mortality
SPECIES DESCRIPTION
Species Code
Common Name
Latin Name
Family Name
LEGEND
Attribute
Multi-Value Attribute
Key Attribute
Composite Multi-Value Key Attribute
Strong Entity
Weak Entity
Relationship
Nicholas Davis
ncdavis2
8/31/2011